



**INSTITUTO
FEDERAL**

Goiás

Câmpus
Formosa

Algoritmos e Estruturas de Dados

Tecnólogo em Análise e Desenvolvimento de Sistemas

Prof. Dr. Waldeyr Mendes Cordeiro da Silva

2019

Material didático para Algoritmos e Estruturas de Dados

Versão 0.1

Autor: Waldeyr Mendes Cordeiro da Silva

- Formação:
 - Técnico em Processamento de Dados
 - Bacharelado em Sistemas de Informação
 - Licenciatura em Ciências Biológicas
 - Complementação Pedagógica em Matemática
 - Especialização em Engenharia de Software
 - Especialização em Segurança da Informação
 - Mestrado em Informática
 - Doutorado em Ciências Biológicas (Bioinformática)
-  Lattes: <http://lattes.cnpq.br/2391349697609978>
-  ORCID: <https://orcid.org/0000-0002-8660-6331>
-  Scopus: <https://www.scopus.com/authid/detail.uri?authorId=57190660358>
-  Google Acadêmico: <https://scholar.google.com.br/citations?user=5gh-liIAAAAJ>

Sumário

1	Prefácio	5
2	Introdução	6
2.1	Análise de Algoritmos	7
2.1.1	Elementos de Notação Assintótica	7
3	Programação	10
3.1	Primeiros Passos em Programação	11
3.2	Trabalhando com Variáveis	12
3.3	Estruturas de Decisão	16
3.3.1	If/Else	16
4	Estruturas de Dados	18
4.1	Estruturas de Dados Homogêneas e Heterogêneas	19
4.2	Listas Encadeadas	22
4.3	Pilhas	27
4.4	Filas	27
4.5	Algoritmos de Ordenação	27
4.6	Árvores	38
4.7	Tabelas Hashing	39

4.8	Grafos	39
4.8.1	Grafos como objetos (Programação orientada a objetos)	42
4.8.2	Busca em Grafos	42
4.8.3	Caminho de Euler	43
4.8.4	Caminho Hamiltoniano	44
5	Aplicações	46
6	Apêndice	49

A globe is centered in the image, partially obscured by a semi-transparent rectangular box on the left. The globe is covered in a network of white lines connecting small white dots, representing a global network or data structure. The background is a deep space scene with a dense field of stars and a bright, glowing nebula or galaxy on the left side. The overall color palette is dark blue and black, with highlights from the stars and the network nodes.

1. Prefácio

Este livro destina-se aos acadêmicos e demais interessados em iniciar os estudos em algoritmos e estruturas de dados.



2. Introdução

Um bonita citação...

Um algoritmo é um procedimento computacional bem definido que processa um dado ou um conjunto de dados (entrada) e produz algum novo dado ou conjunto de dados (saída) (CORMEN et al., 2009). Os algoritmos existem há muito tempo, mas estão presentes na sociedade moderna de uma forma nunca experimentada nestes cerca de 2,5 milhões de anos da história humana. Quase tudo que se produz, sejam produtos ou serviços, tem alguma influência de algoritmos. O comércio eletrônico utiliza tanto algoritmos clássicos como ordenações, quanto algoritmos modernos de recomendação de produtos baseado no histórico de visitas. A segurança das senhas em sistemas *Web* ou bancários é baseada em algoritmos de criptografia. Imagens de satélite, dados genômicos, reconhecimento de faces, previsão do tempo, quase tudo que se possa imaginar atualmente é influenciado direta ou indiretamente por algum algoritmo.

O estudo de algoritmos é uma demanda crescente frente aos novos desafios trazidos pelo grande volume de dados que as tecnologias modernas proporcionaram. A análise de algoritmos ainda tem questões importantes em aberto. A mais conhecida é o problema *P versus NP*, que pergunta se todo problema cuja solução pode ser verificada rapidamente também pode ser resolvido rapidamente (COOK, 1971). Ele integra a lista dos sete *Millennium Prize Problems* do

Clay Mathematics Institute, que oferece 1 milhão de dólares pela solução de cada um ([CLAY MATHEMATICS INSTITUTE, 2000](#)).

2.1 Análise de Algoritmos

O propósito da análise de um algoritmo é prever seu comportamento quanto ao tempo de execução ou ao espaço em memória que irá ocupar mesmo antes de ser executado em um computador específico ([MANBER, 1989](#)). Muitos fatores, especialmente o tamanho e a variedade dos dados de entrada, influenciam o comportamento do algoritmo. Portanto, a análise de algoritmos provê uma aproximação, o que em muitos casos é suficiente para comparar soluções. As possibilidades de entradas diferentes, ainda que de mesmo tamanho são muito vastas e por isso um algoritmo não se comportará da mesma maneira para diferentes entradas de um mesmo tamanho. Por esta razão a análise de algoritmos se concentra primordialmente em predizer o tempo aproximado de execução baseada no tamanho da entrada, tirando da equação elementos como o hardware, por exemplo. Assim, ainda que haja uma variação no tempo de execução dependente da variabilidade da entrada, essa variação tem menor importância do que o tamanho da entrada.

Para entradas de tamanho (n) pequeno, qualquer algoritmo custa pouco para ser executado, portanto, a análise de algoritmos concentra-se em valores grandes de n ([ZIVIANI, 2007](#)). O comportamento assintótico de uma função $f(n)$ representa o limite do comportamento do custo quando n cresce.

2.1.1 Elementos de Notação Assintótica

Melhor caso, pior caso e caso médio

Para entradas de um mesmo tamanho n , o custo de um algoritmo pode variar conforme a disposição dos dados. Vejamos um exemplo com a busca sequencial, que lê os valores em ordem até encontrar o procurado. Tomando os valores 7, 2, 5, 4, 9 dispostos desta maneira, encontraríamos o número 9 em cinco passos. Em outra entrada de mesmo tamanho, mas com os valores dispostos de outra maneira, 5, 9, 7, 4, 2, encontraríamos o número 9 em dois passos. Independentemente das permutações possíveis, encontraríamos o número procurado em no máximo $n = 5$ passos (**pior caso**) e em no mínimo 1 passo (**melhor caso**), quando ele está na primeira posição. Se todas as posições forem igualmente prováveis, a busca faz em média $(n + 1)/2$ comparações (**caso médio**). O pior caso é o mais usado na prática, pois garante um limite para qualquer entrada de tamanho n ([CORMEN et al., 2009](#)).

Notação Big O

A notação O denota um **limite assintótico superior** para uma função: $f(n) = O(g(n))$ significa que, a partir de algum tamanho de entrada, $f(n)$ não cresce mais rápido do que $g(n)$, a menos de uma constante. É comum usar O para descrever o tempo de pior caso, mas limite superior e pior caso são conceitos diferentes: é possível, por exemplo, dar um limite superior para o melhor caso. A busca sequencial acima executa em $O(n)$ no pior caso.

Na análise de algoritmos, as constantes, associadas ou não ao tamanho da entrada, têm baixa representatividade no tempo de execução ([MANBER, 1989](#)). Por exemplo, se um algoritmo executa em $2n^2 + 50$, dizemos que o algoritmo executa em n^2 pois as constantes 2 e 50 não interferem significativamente no crescimento assintótico da função. O tempo de execução em função do tamanho da entrada está relacionado às várias classes de problemas que têm as seguintes funções de complexidade:

- $f(n) = O(1) \mapsto$ Complexidade constante. O tempo de execução independe do tamanho (n) da entrada.
- $f(n) = O(\log n) \mapsto$ Complexidade logarítmica. O tempo cresce muito lentamente: dobrar o tamanho da entrada acrescenta apenas um passo. Tipicamente ocorre em algoritmos que

descartam uma fração fixa da entrada a cada passo, como a busca binária.

- $f(n) = O(n) \mapsto$ Complexidade linear. Tempo de execução linear ao tamanho da entrada. São, em geral, algoritmos que realizam uma quantidade constante de trabalho para cada elemento da entrada.
- $f(n) = O(n \log n) \mapsto$ Essa complexidade ocorre normalmente em algoritmos que quebram o problema em partes menores, solucionam-nas de forma independente e juntam em seguida essas subsoluções, como o *Merge Sort*.
- $f(n) = O(n^2) \mapsto$ Complexidade quadrática. O crescimento quadrático significa que o tempo de execução crescerá rapidamente, pois será proporcional ao quadrado do tamanho da entrada. Ou seja, para entradas grandes, o algoritmo levará muito tempo para ser executado.
- $f(n) = O(n^3) \mapsto$ Complexidade cúbica. Assim como no algoritmo de complexidade quadrática, o tempo será proporcional ao cubo do tamanho da entrada. Algoritmos desse tipo são úteis apenas para pequenas tarefas.
- $f(n) = O(2^n) \mapsto$ Complexidade exponencial. Algoritmos desse tipo geralmente não são úteis do ponto de vista prático. Geralmente são soluções baseadas em abordagens de “força bruta”.
- $f(n) = O(n!) \mapsto$ Complexidade fatorial. São ainda piores em termos de tempo de execução do que os algoritmos exponenciais.

Geralmente não é uma tarefa trivial determinar se uma certa função $f(n)$ é $O(g(n))$. Formalmente, uma função $g(n)$ domina assintoticamente outra função $f(n)$ se existem constantes positivas c e n_0 tais que $|f(n)| \leq c \cdot |g(n)|$ para todo $n \geq n_0$ (ZIVIANI, 2007; CORMEN et al., 2009). Nesse caso, escreve-se $f(n) = O(g(n))$. Ou seja, para todos os valores de n a partir de n_0 , o valor de $f(n)$ fica abaixo de $g(n)$ multiplicada por uma constante c . Para ilustrar o domínio assintótico de uma função, considere as funções $g(n) = 6n^2$ e $f(n) = 5n^2 + 15$. Com $c = 1$ e $n_0 = 4$, tem-se $f(n) \leq g(n)$ para todo $n \geq 4$, como mostra a Figura 2.1; logo, $5n^2 + 15 = O(n^2)$.

$$g(n) = \begin{cases} n=1 & 6 \cdot 1^2 = 6 \\ n=2 & 6 \cdot 2^2 = 24 \\ n=3 & 6 \cdot 3^2 = 54 \\ n=4 & 6 \cdot 4^2 = 96 \\ n=5 & 6 \cdot 5^2 = 150 \\ n=6 & 6 \cdot 6^2 = 216 \\ n=7 & 6 \cdot 7^2 = 294 \\ \dots & \end{cases} \quad f(n) = \begin{cases} n=1 & 5 \cdot 1^2 + 15 = 20 \\ n=2 & 5 \cdot 2^2 + 15 = 35 \\ n=3 & 5 \cdot 3^2 + 15 = 60 \\ n=4 & 5 \cdot 4^2 + 15 = 95 \\ n=5 & 5 \cdot 5^2 + 15 = 140 \\ n=6 & 5 \cdot 6^2 + 15 = 195 \\ n=7 & 5 \cdot 7^2 + 15 = 260 \\ \dots & \end{cases}$$

Notações Ω e Θ

De forma análoga, a notação Ω denota um **limite assintótico inferior**: $f(n) = \Omega(g(n))$ se existem constantes positivas c e n_0 tais que $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$. Já a notação Θ denota um **limite assintótico firme**: $f(n) = \Theta(g(n))$ se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$, ou seja, se $f(n)$ fica entre $c_1 \cdot g(n)$ e $c_2 \cdot g(n)$ para todo $n \geq n_0$ (KNUTH, 1976; CORMEN et al., 2009). No exemplo anterior, $5n^2 + 15 = \Theta(n^2)$, pois também vale $5n^2 + 15 \geq 5n^2$ para todo $n \geq 1$. A Tabela 2.1 resume as três notações.

Diz-se que uma função $f(n)$ cresce monotonamente se $n_1 \geq n_2$ implica $f(n_1) \geq f(n_2)$. Funções exponenciais crescem mais rápido do que funções polinomiais, ou seja, para as con-

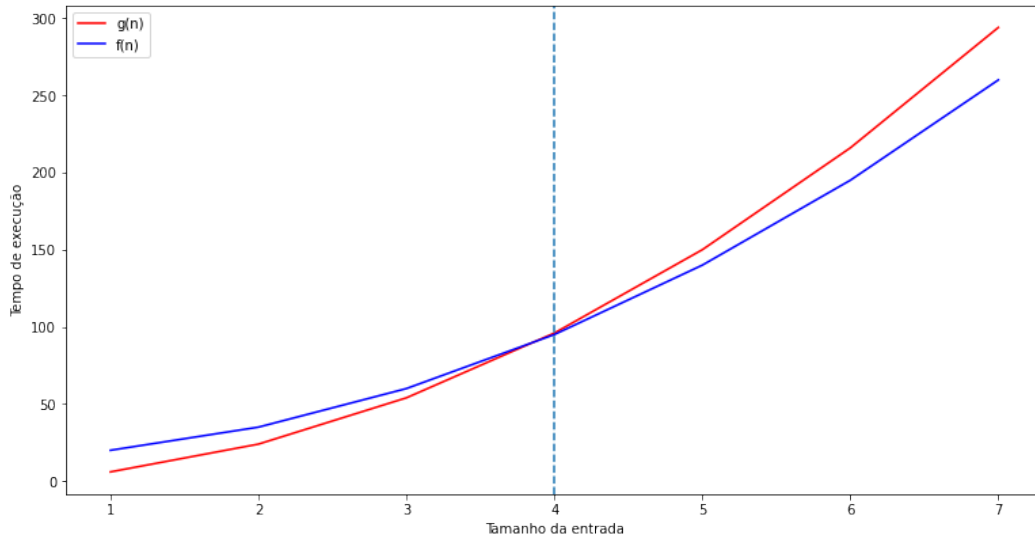
Fig. 2.1: Domínio assintótico da função $g(n)$ sobre a função $f(n)$ quando $n \geq 4$.

Tabela 2.1: Notações assintóticas.

Notação	Significado	Definição (para todo $n \geq n_0$)
$f(n) = O(g(n))$	f não cresce mais rápido que g	$0 \leq f(n) \leq c g(n)$
$f(n) = \Omega(g(n))$	f não cresce mais devagar que g	$0 \leq c g(n) \leq f(n)$
$f(n) = \Theta(g(n))$	f cresce na mesma taxa que g	$c_1 g(n) \leq f(n) \leq c_2 g(n)$

stantes $c > 0$ e $a > 1$, e para todas as funções $f(n)$ de crescimento monótono:

$$(f(n))^c = O(a^{f(n)})$$

Esta regra pode ser usada para comparar várias funções. Por exemplo, substituindo $f(n)$ por n temos:

$$n^c = O(a^n)$$

A Tabela 2.2 ilustra os tempos de execução para diferentes tamanhos de entrada em complexidades polinomiais e exponenciais.

Tabela 2.2: Comparação de tempos de execução polinomiais e exponenciais assumindo que 0,000001s é o tempo gasto para um $n = 1$. Adaptado de [Garey and Johnson \(1979\)](#).

Complexidade	Tamanho da entrada (n)					
	10	20	30	40	50	60
n	0,00001s	0,00002s	0,00003s	0,00004s	0,00005s	0,00006s
n^2	0,0001s	0,0004s	0,0009s	0,0016s	0,0025s	0,0036s
n^3	0,001s	0,008s	0,027s	0,064s	0,125s	0,216s
2^n	0,001s	1,0s	17,9 min	12,7 dias	35,7 anos	366 séculos

3. Programação

3.1 Primeiros Passos em Programação

Dennis Ritchie criou a linguagem “C” em 1972 quando trabalhava nos laboratórios da Bell Telephone. A linguagem C foi criada para desenvolver uma nova versão do sistema operacional Unix que utilizava Assembly até então. Inicialmente, a cada nova implementação de Unix para um tipo de máquina, um novo compilador C devia ser desenvolvido para essa máquina. Nos anos 80, C tornou-se popular fora do ambiente Unix, criando a demanda por compiladores comerciais. Desde então, C é considerada uma linguagem de propósito geral, com instruções de alto nível e capaz de gerar programas rápidos em tempo de execução. Além disso, C influenciou de forma direta a sintaxe e estruturas de linguagens modernas como C++, Java, C#, Objective C, e muitas outras.

A linguagem C foi padronizada pelo *American National Standards Institute* (ANSI) em 1989 (C89) para garantir a compatibilidade e a portabilidade da linguagem. O padrão foi adotado pela *International Organization for Standardization* (ISO) em 1990 e revisado em 1999 (C99), 2011 (C11), 2018 (C17) e 2024 (C23). Os exemplos deste livro seguem o padrão C17 (ISO, 2018).

Um código escrito em linguagem C funciona com comandos, que são normalmente palavras em inglês (sintaxe) e são organizados em determinadas estruturas. Uma vez escritos em um arquivo, os comandos são então lidos pelos compiladores e convertidos para a linguagem de máquina que corresponde a ação pretendida. O Programa 3.1 mostra um exemplo de programa em C.

Programa 3.1: Meu primeiro programa em C.

```
1 #include <stdio.h>
2 int main(void) {
3     printf("Meu primeiro programa em C!\n\n");
4     return 0;
5 }
```

Após escrito, o programa precisa ser compilado. Se o programa tiver sido escrito corretamente (sintaxe e estrutura), o compilador gera um executável. É importante perceber que compiladores não verificam erros lógicos, ou seja de concepção do código. É justamente aí que reside o valor do trabalho humano como programador: combinar sintaxe e estruturas de forma que solucionem um dado problema do mundo real. Caso o compilador encontre algum erro no código dois tipos de mensagens podem ser emitidas:

1. Erro (*error*): Quando o código tem um problema que impede o compilador de gerar um executável
2. Aviso (*warning*): Quando o problema no código não impede de gerar um executável, mas requer algum tipo de atenção.

Para compilar o Programa 3.1 em Linux com o compilador GCC (as instruções de instalação estão no arquivo README.md do repositório) basta o seguinte comando no terminal:

```
gcc -Wall prog001.c -o prog001
```

A opção `-Wall` pede ao compilador que mostre todos os avisos comuns, o que ajuda a encontrar erros. Após esta ação, será gerado o arquivo executável `prog001`, o qual poderá ser executado com o comando:

```
./prog001
```

Todo programa em C começa pela função `main`, que devolve um inteiro ao sistema operacional: por convenção, `return 0`; indica que o programa terminou sem erros.

No Programa 3.1 é impressa uma mensagem na tela. O caractere especial representado por “\n” identifica uma quebra de linha. Essa mensagem é representada como uma cadeia de caracteres (*string*) limitada pelas aspas. Se for necessário representar as aspas em uma *string*, será necessário escapá-las usando a barra “\”. O Programa 3.2 mostra como fazer isso.

Programa 3.2: Escapando aspas.

```

1 #include <stdio.h>
2 int main(void) {
3     printf("Que lindo dia para aprender \"C\"\\n");
4     return 0;
5 }

```

3.2 Trabalhando com Variáveis

Uma variável é um objeto capaz de guardar e representar um valor ou expressão em memória. Às variáveis são associadas a nomes ou identificadores e sua existência é limitada por um espaço e um tempo na memória do computador, o qual frequentemente é igual ao tempo de execução do programa em que elas foram criadas.

Nas linguagens de programação é comum existirem tipos de dados predefinidos, chamados tipos primitivos. Em C, os tipos primitivos são:

- char: caractere
- int: inteiro
- float: real de precisão simples
- double: real de precisão dupla
- void: vazio ou sem valor

Há ainda os modificadores que alteram o sinal ou o tamanho, e consequentemente a capacidade, dos tipos em memória. São eles:

- signed
- unsigned
- long
- short

O padrão da linguagem define apenas os tamanhos **mínimos** de cada tipo; os tamanhos reais dependem do compilador e da arquitetura (ISO, 2018). A Tabela 3.1 combina os tipos e os modificadores e mostra os valores típicos do GCC em Linux de 64 bits (modelo LP64). Os limites exatos de cada sistema estão nas constantes de `<limits.h>` (por exemplo, `INT_MAX`) e `<float.h>`, e o operador `sizeof` informa o tamanho em *bytes* de um tipo ou variável. Quando o tamanho exato importa, podem ser usados os tipos de `<stdint.h>`, como `int32_t` e `uint64_t`.

Tabela 3.1: Tipos primitivos e modificadores em C: valores típicos do GCC em Linux de 64 bits.

Palavra-chave	Tipo	Tamanho (Bytes)	Intervalo
char	Caractere	1	-128 a 127 ou 0 a 255*
signed char	Caractere com sinal	1	-128 a 127
unsigned char	Caractere sem sinal	1	0 a 255
short int	Inteiro curto	2	-32.768 a 32.767
unsigned short int	Inteiro curto sem sinal	2	0 a 65.535
int	Inteiro	4	-2.147.483.648 a 2.147.483.647
unsigned int	Inteiro sem sinal	4	0 a 4.294.967.295
long int	Inteiro longo	8	$-9,22 \times 10^{18}$ a $9,22 \times 10^{18}$
unsigned long int	Inteiro longo sem sinal	8	0 a $1,84 \times 10^{19}$
long long int	Inteiro longo longo	8	$-9,22 \times 10^{18}$ a $9,22 \times 10^{18}$
float	Ponto flutuante com precisão simples	4	$\pm 1,2 \times 10^{-38}$ a $\pm 3,4 \times 10^{38}$
double	Ponto flutuante com precisão dupla	8	$\pm 2,2 \times 10^{-308}$ a $\pm 1,8 \times 10^{308}$
long double	Ponto flutuante com precisão estendida	16	$\pm 3,4 \times 10^{-4932}$ a $\pm 1,2 \times 10^{4932}$

* O char simples pode ter ou não sinal, conforme o compilador e a arquitetura; no GCC para x86-64, ele tem sinal. Os tipos `int`, `short int` e `long int` já têm sinal; por isso `signed`

`int` equivale a `int`. Em Windows, `long int` costuma ter 4 *bytes*. Para os tipos de ponto flutuante, os intervalos indicam o menor e o maior valor absoluto normalizado.

O Programa 3.3 mostra um exemplo de variável representando o valor inteiro 10 e em seguida a impressão de uma mensagem com seu valor. Na impressão, é importante notar que a mensagem é formatada para que em um determinado ponto (`%d`) apareçam os dígitos correspondentes ao valor de `a`, que é a variável que abriga o valor 10.

Programa 3.3: Trabalhando com variáveis.

```
1 #include <stdio.h>
2 int main(void) {
3     int a = 10;
4     printf("O valor de a = %d\n", a);
5     return 0;
6 }
```

As formas de declarar variáveis podem ser diversas. O Programa 3.4 mostra a declaração de variáveis `n1` e `n2` atribuindo imediatamente os valores, enquanto a variável `soma` é criada sem que seu valor seja atribuído imediatamente. No Programa 3.5, as variáveis `n1` e `n2` são declaradas na mesma linha, separadas por vírgula, uma vez que são do mesmo tipo `int`.

Programa 3.4: Trabalhando com variáveis.

```
1 #include <stdio.h>
2 int main(void) {
3     int n1 = 37;
4     int n2 = -37;
5     int soma;
6     soma = n1 + n2;
7     printf("%d + %d = %d \n", n1, n2, soma);
8     return 0;
9 }
```

Programa 3.5: Trabalhando com variáveis.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int n1 = 1, n2 = 2;
5     int soma = n1 + n2;
6     printf("%d + %d = %d \n", n1, n2, soma);
7     return 0;
8 }
```

Quanto à atribuição de valores a uma variável, ela pode ser direta ou indireta, ou seja, um valor pode ser diretamente atribuído a uma variável ou o valor de uma variável pode ser atribuído a outra, como no exemplo do Programa 3.6, onde `n1` recebe o valor de `n2` que recebe o valor de `n3` que recebe 10. Nesse caso, `n1`, `n2` e `n3` terão valor = 10.

Programa 3.6: Trabalhando com variáveis.

```
1 #include <stdio.h>
2
3 int main(void) {
4     int n1;
5     int n2;
6     int n3;
7     n1 = n2 = n3 = 10;
8     printf("n1 = %d \n", n1);
9     printf("n2 = %d \n", n2);
10    printf("n3 = %d \n", n3);
11    return 0;
12 }
```

Dependendo do tipo primitivo de dados da variável, diferentes formatações podem ser usadas na função `printf()` conforme pode ser visto na Tabela 3.2. O Programa 3.7 ilustra um exemplo de impressão de um `float` com 2 casas decimais.

Programa 3.7: Trabalhando com variáveis.

```

1 #include <stdio.h>
2 int main(void) {
3     float pi = 3.1415;
4     printf("Pi = %.2f\n", pi);
5     return 0;
6 }
```

Tabela 3.2: Formatação da saída de dados com a função `printf()`.

Formatação	Significado
<code>%c</code>	caractere
<code>%d</code>	inteiro na base 10
<code>%i</code>	inteiro na base 10
<code>%ld</code>	inteiro longo (<code>long int</code>)
<code>%u</code>	inteiro sem sinal (<i>unsigned</i>)
<code>%zu</code>	tamanho (<code>size_t</code>), resultado de <code>sizeof</code>
<code>%f</code>	ponto flutuante (<code>float</code> ou <code>double</code>)
<code>%e</code>	ponto flutuante em notação exponencial
<code>%Lf</code>	ponto flutuante long double
<code>%o</code>	octal (base 8)
<code>%x</code>	hexadecimal (base 16)
<code>%s</code>	<i>string</i>
<code>%p</code>	endereço de memória (ponteiro)
<code>%%</code>	caractere de porcentagem

O Programa 3.8 mostra algumas variáveis e seus respectivos tamanhos em *Bytes* com sua saída formatada.

Programa 3.8: Trabalhando com variáveis.

```

1 #include <stdio.h>
2 int main(void) {
3     signed char v1 = 'A';
4     int v2 = 2147483647;
5     short v3 = 32767;
6     long int v4 = 2147483647L;
7     float v5 = 3.4E38f;
8     double v6 = 1.7E+308;
9     long double v7 = 1.79769e+308L;
10    // sizeof devolve um valor do tipo size_t, impresso com %zu
11    printf("Tamanho de %c = %zu bytes \n", v1, sizeof(v1));
12    printf("Tamanho de %d = %zu bytes \n", v2, sizeof(v2));
13    printf("Tamanho de %hd = %zu bytes \n", v3, sizeof(v3));
14    printf("Tamanho de %ld = %zu bytes \n", v4, sizeof(v4));
15    printf("Tamanho de %.1e = %zu bytes \n", v5, sizeof(v5));
16    printf("Tamanho de %.1e = %zu bytes \n", v6, sizeof(v6));
17    printf("Tamanho de %.1Le = %zu bytes \n", v7, sizeof(v7));
18    return 0;
19 }
```

Operadores Aritméticos

Os operadores aritméticos usados em C são mostrados na Tabela 3.3. Quando uma expressão

Tabela 3.3: Operadores aritméticos.

Operador	Sintaxe
Adição unária	+a
Adição	a + b
Incremento pré-fixado	++a
Incremento pós-fixado	a++
Atribuição por adição	a += b
Subtração unária	-a
Subtração	a - b
Decremento pré-fixado	--a
Decremento pós-fixado	a--
Atribuição por subtração	a -= b
Multiplicação	a * b
Atribuição por multiplicação	a *= b
Divisão	a / b
Atribuição por divisão	a /= b
Módulo (resto)	a % b
Atribuição por módulo (resto)	a %= b

tem vários operadores, a ordem de avaliação segue a precedência mostrada na Tabela 3.4; em caso de dúvida, os parênteses deixam a ordem explícita. Em C não há operador de potência: usa-se a função `pow()` da biblioteca `<math.h>`. O Programa 3.9 ilustra o uso do operador %

Tabela 3.4: Precedência dos principais operadores em C (da maior para a menor).

Prioridade	Operadores	Descrição
1º	() [] -> . a++ a--	chamada, índice, campo, pós-fixados
2º	++a --a +a -a ! *p &a sizeof	unários
3º	* / %	multiplicativos
4º	+ -	aditivos
5º	< <= > >=	relacionais
6º	== !=	igualdade
7º	&&	E lógico
8º		OU lógico
9º	?:	condicional (ternário)
10º	= += -= *= /= %=	atribuição

(módulo), que retorna o resto da divisão inteira. Vale lembrar que a divisão entre dois inteiros em C descarta a parte fracionária: $7 / 2$ resulta em 3, enquanto $7.0 / 2$ resulta em 3.5.

Programa 3.9: Trabalhando com variáveis.

```

1 #include <stdio.h>
2
3 int main(void) {
4     // Operators
5     // + soma
6     // - subtracao
7     // * multiplicacao
8     // / divisao
9     // % modulo (divisao inteira)

```

```
10     int n1 = 21;
11     int m = 21 % 4;
12     printf("Modulo de %d = %d \n", n1, m);
13     return 0;
14 }
```

Entrada de Dados

A maneira mais clássica de um programa em C obter dados digitados pelo usuário através do *prompt* é usando a função *scanf()*. O Programa 3.10 mostra um exemplo onde os valores das variáveis *a* e *b* são informados pelo usuário e depois somados.

Programa 3.10: Trabalhando com a função *scanf()* para entrada de dados em C.

```
1  #include <stdio.h>
2  int main(void) {
3      int a, b, soma;
4
5      printf("Digite um numero inteiro: ");
6      scanf("%d", &a);
7
8      printf("Digite um numero inteiro: ");
9      scanf("%d", &b);
10
11     soma = a + b;
12     printf("O valor da soma: %d\n", soma);
13     return 0;
14 }
```

3.3 Estruturas de Decisão

3.3.1 If/Else

O *if* é utilizado para testar uma proposição lógica. Em C, qualquer valor diferente de zero é considerado verdadeiro, e zero é considerado falso. A proposição é escrita entre parênteses. Caso o resultado seja verdadeiro, o bloco de instruções subordinadas ao *if* será executado. O bloco é delimitado por “{” e “}”. Se houver apenas uma instrução subordinada, não há necessidade das chaves.

O *else* é utilizado para capturar a condição não atendida no teste do *if*, o “caso contrário”. O Programa 3.11 mostra um exemplo onde o valor da variável *x* é testado para saber se é maior ou igual a zero. Através deste teste é possível identificar se o valor é positivo (*if* verdadeiro $\rightarrow$ *valor* $\geq$ 0), ou é negativo (*else* $\rightarrow$ *valor* $<$ 0).

Programa 3.11: Trabalhando com o *if* em C.

```
1  #include <stdio.h>
2  int main(void) {
3      int x;
4      printf("Informe um valor inteiro: ");
5      scanf("%d", &x);
6      if (x>=0){
7          printf("%d: positivo.\n", x);
8      }
9      else{
10         printf("%d: negativo.\n", x);
11     }
12     return 0;
13 }
```

Há ainda a possibilidade de verificar múltiplas condições com o *else if*. O uso do *else if* é demonstrado no Programa 3.12 que verifica se um valor inteiro informado pelo usuário é igual a 10, 20 ou 30.

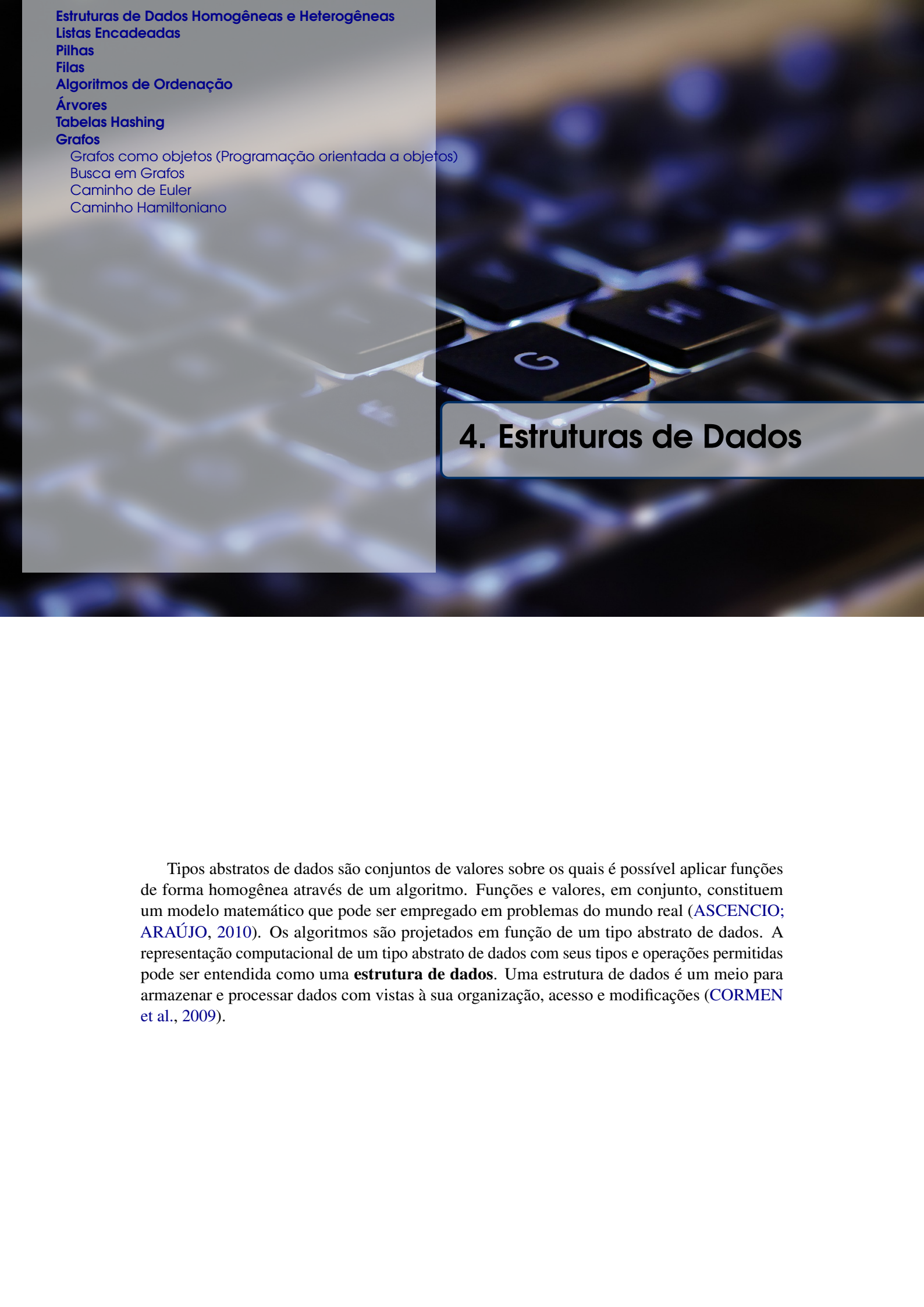
Programa 3.12: Trabalhando com o *if*, *else if*, *else* em C.

```
1 #include <stdio.h>
2 int main(void) {
3     int x;
4     printf("Informe um valor inteiro: ");
5     scanf("%d", &x);
6     if (x==10){
7         printf("%d.\n", x);
8     }
9     else if (x==20){
10        printf("%d.\n", x);
11    }
12    else if (x==30){
13        printf("%d.\n", x);
14    }
15    else{
16        printf("Diferente de 10, 20 ou 30.\n");
17    }
18    return 0;
19 }
```

Um exemplo onde caracteres podem ser usados é mostrado no Programa 3.13. Neste programa uma questão apresenta alternativas *a*, *b*, *c*, *d* e *e*. O usuário escolhe a letra correspondente à resposta correta e o programa diz se ele acertou ou não. Além disso, reporta se a opção escolhida é inválida.

Programa 3.13: Trabalhando com o *if*, *else if*, *else* em C.

```
1 #include <stdio.h>
2 int main(void) {
3     printf ("Quem descobriu o Brasil?\n\n");
4     printf ("a) Cabral\n");
5     printf ("b) Cabrel\n");
6     printf ("c) Cabril\n");
7     printf ("d) Cabrol\n");
8     printf ("e) Cabrul\n\n");
9     char resposta;
10    printf("Resposta (a,b,c,d ou e): ");
11    scanf("%c", &resposta);
12    if (resposta=='a'){
13        printf("Certo.\n");
14    }
15    else if (resposta=='b'){
16        printf("Errado.\n");
17    }
18    else if (resposta=='c'){
19        printf("Errado.\n");
20    }
21    else if (resposta=='d'){
22        printf("Errado.\n");
23    }
24    else if (resposta=='e'){
25        printf("Errado.\n");
26    }
27    else{
28        printf("Opcao invalida.\n");
29    }
30    return 0;
31 }
```



- Estruturas de Dados Homogêneas e Heterogêneas
- Listas Encadeadas
- Pilhas
- Filas
- Algoritmos de Ordenação
- Árvores
- Tabelas Hashing
- Grafos
 - Grafos como objetos (Programação orientada a objetos)
 - Busca em Grafos
 - Caminho de Euler
 - Caminho Hamiltoniano

4. Estruturas de Dados

Tipos abstratos de dados são conjuntos de valores sobre os quais é possível aplicar funções de forma homogênea através de um algoritmo. Funções e valores, em conjunto, constituem um modelo matemático que pode ser empregado em problemas do mundo real ([ASCENCIO; ARAÚJO, 2010](#)). Os algoritmos são projetados em função de um tipo abstrato de dados. A representação computacional de um tipo abstrato de dados com seus tipos e operações permitidas pode ser entendida como uma **estrutura de dados**. Uma estrutura de dados é um meio para armazenar e processar dados com vistas à sua organização, acesso e modificações ([CORMEN et al., 2009](#)).

4.1 Estruturas de Dados Homogêneas e Heterogêneas

Vetores

Primeiro, uma revisão sobre vetores. Um vetor é uma sequência de elementos do mesmo tipo, guardados em posições contíguas de memória e acessados por um índice que, em C, começa em zero. Em C, o nome de um vetor, quando usado em uma expressão, é convertido em um ponteiro para o seu primeiro elemento. Portanto, quando um vetor é parâmetro para uma função, o que é passado é o endereço base do vetor, e a função pode alterar os elementos originais. Por essa mesma razão, a função não conhece o tamanho do vetor, que precisa ser passado como um parâmetro à parte. Vetores podem ser uni-, bi-, tri- ou multidimensionais, mas abrigam sempre o mesmo tipo de dados. O Programa 4.1 mostra o vetor sendo passado para uma função que retorna o valor do meio do vetor, e o Programa 4.2 mostra um vetor de duas dimensões (matriz).

Programa 4.1: Em C, vetores são passados para funções por referência.

```
1 #include <stdio.h>
2 #define TAMANHO 100
3 int getValorDoMeio(int *vetor, int tamanho);
4
5 int main(void) {
6     int vetor[TAMANHO];
7     int valorDoMeio;
8     for(int i=0; i<TAMANHO;i++){
9         vetor[i] = i+1;
10    }
11    valorDoMeio = getValorDoMeio(vetor,TAMANHO);
12    printf("Valor do meio: %d \n", valorDoMeio);
13    return 0;
14 }
15
16 int getValorDoMeio(int *vetor, int tamanho){
17     printf("Valores no vetor:\n");
18     for(int i=0; i<tamanho;i++){
19         printf("%d\t",vetor[i]);
20     }
21     printf("\n");
22     return vetor[tamanho/2];
23 }
24 // Valores no vetor:
25 // 1   2   3   4   5   6   7   8   9   10
26 // 11  12  13  14  15  16  17  18  19  20
27 // 21  22  23  24  25  26  27  28  29  30
28 // 31  32  33  34  35  36  37  38  39  40
29 // 41  42  43  44  45  46  47  48  49  50
30 // 51  52  53  54  55  56  57  58  59  60
31 // 61  62  63  64  65  66  67  68  69  70
32 // 71  72  73  74  75  76  77  78  79  80
33 // 81  82  83  84  85  86  87  88  89  90
34 // 91  92  93  94  95  96  97  98  99  100
35 // Valor do meio: 51
```

Programa 4.2: Um vetor de duas dimensões mostrando apenas valores da diagonal principal.

```
1 #include <stdio.h>
2 #define TAMANHO 4
3 int main(void) {
4     int vetor[TAMANHO][TAMANHO];
5     for (int l=0; l<TAMANHO; l++){
6         for (int c=0; c<TAMANHO; c++){
7             printf("Valor [%d][%d]: ", l, c);
8             scanf("%d", &vetor[l][c]);
9         }
10    }
11
12    for (int l=0; l<TAMANHO; l++){
13        for (int c=0; c<TAMANHO; c++){
```

```

14         if(l == c){
15             printf("[%d][%d]:%d\t",l,c,vetor[l][c]);
16         }
17     }
18     printf("\n");
19 }
20 return 0;
21 }
22 // Valor [0][0]: 1
23 // Valor [0][1]: 2
24 // Valor [0][2]: 2
25 // Valor [0][3]: 2
26 // Valor [1][0]: 2
27 // Valor [1][1]: 1
28 // Valor [1][2]: 2
29 // Valor [1][3]: 2
30 // Valor [2][0]: 2
31 // Valor [2][1]: 2
32 // Valor [2][2]: 1
33 // Valor [2][3]: 2
34 // Valor [3][0]: 2
35 // Valor [3][1]: 2
36 // Valor [3][2]: 2
37 // Valor [3][3]: 1
38 // [0][0]:1
39 // [1][1]:1
40 // [2][2]:1
41 // [3][3]:1

```

Strings em C

Em C, uma *string* é um vetor de caracteres terminado pelo caractere nulo `'\0'`. Uma constante *string* é definida entre aspas e o caractere nulo é incluído automaticamente ao seu final. Por exemplo, a *string* `"IFG"` é um vetor de 4 elementos: `'I'`, `'F'`, `'G'` e `'\0'`. Por isso, um vetor que guarda uma *string* precisa ter uma posição a mais do que a quantidade de caracteres. O Programa 4.3 mostra um exemplo de *string* em C que percorre os caracteres até encontrar o caractere nulo.

Programa 4.3: Em C, *strings* são vetores de caracteres.

```

1  #include <stdio.h>
2  #define TAMANHO 100
3  int main(void) {
4      char vetor[TAMANHO] = "Aqui vai uma frase bastante longa para servir de exemplo!";
5      int qtd_de_letras_a_na_string = 0;
6      // percorre a string ate encontrar o caractere nulo '\0' que marca o seu fim
7      for (int i = 0; vetor[i] != '\0'; i++) {
8          if (vetor[i] == 'a' || vetor[i] == 'A') {
9              qtd_de_letras_a_na_string++;
10         }
11     }
12     printf("Quantidade de letras \'a\' ou \'A\' na string: %d\n",
13           qtd_de_letras_a_na_string);
14     return 0;
15 }
16 // Quantidade de letras 'a' ou 'A' na string: 8

```

Structs

Em C, uma *struct* é uma forma de criar novos tipos heterogêneos, agrupando campos de tipos diferentes sob um mesmo nome. O Programa 4.4 mostra um exemplo de *struct* em C. Nele, a função `imprimirPessoa` recebe a *struct* por valor (uma cópia), enquanto `setPessoa` recebe um ponteiro e, por isso, altera a variável original.

Programa 4.4: Structs em C.

```
1 #include <stdio.h>
2 #include <string.h>
3 #define CPF_TAM 15 // 14 caracteres do CPF formatado + 1 para o '\0'
4 #define NOME_TAM 100
5
6 typedef struct { // cria um novo tipo como struct
7     float peso; // campo peso
8     int idade; // campo idade
9     float altura; // campo altura
10    char cpf[CPF_TAM]; //campo cpf
11    char nome[NOME_TAM]; //campo nome
12 } Pessoa; // nome do novo tipo criado
13
14 /*Funcao para calcular o IMC*/
15 float calcularIMC(Pessoa p) {
16     return p.peso / (p.altura * p.altura);
17 }
18
19 /*Funcao para imprimir dados da pessoa (parametro por valor)*/
20 void imprimirPessoa(Pessoa p) {
21     printf("CPF: %s\nNome: %s\nIdade: %d\nPeso: %.2f\nAltura: %.2f\n", p.cpf, p.nome,
22           p.idade, p.peso, p.altura);
23     printf("IMC: %.2f\n", calcularIMC(p));
24 }
25
26 /*Funcao para "preencher" uma pessoa (parametro por referencia)*/
27 void setPessoa(Pessoa* p, int idade, float peso, float altura, const char cpf[], const
28               char nome[]) {
29     // Quando usando ponteiros, o campo pode ser acessado de 2 formas:
30     // a) (*nome_do_ponteiro).nome_do_campo
31     // b) nome_do_ponteiro->nome_do_campo
32     (*p).idade = idade; //exemplo a)
33     p->peso = peso; //exemplo b)
34     p->altura = altura;
35     // copia as strings sem ultrapassar o tamanho dos campos e garante o '\0' final
36     snprintf(p->cpf, sizeof(p->cpf), "%s", cpf);
37     snprintf(p->nome, sizeof(p->nome), "%s", nome);
38 }
39
40 int main(void) {
41     Pessoa pessoa01;
42     setPessoa(&pessoa01, 37, 70.0f, 1.75f, "111.111.111-11", "Pessoa da Silva");
43     imprimirPessoa(pessoa01);
44     return 0;
45 }
```

O Programa 4.5 mostra um exemplo de composição de *struct* em C, em que uma *struct* é campo de outra.

Programa 4.5: Composição de Structs em C.

```

1  #include <stdio.h>
2  #include <string.h>
3  #define TAM 100
4  typedef struct { // cria um novo tipo como struct
5      char nome[TAM]; //nome da marca
6      char nacionalidade[TAM]; //nacionalidade da marca
7  } Marca; // nome do novo tipo criado
8
9  typedef struct { // cria um novo tipo como struct
10     char modelo[TAM]; //modelo do carro
11     float motor; //motorizacao
12     Marca marca; //uma struct dentro de outra struct (composicao)
13 } Carro; // nome do novo tipo criado
14
15 void setMarca(Marca* marca, const char nome[], const char nacionalidade[]) {
16     // copia as strings sem ultrapassar o tamanho dos campos e garante o '\0' final
17     snprintf(marca->nome, sizeof(marca->nome), "%s", nome);
18     snprintf(marca->nacionalidade, sizeof(marca->nacionalidade), "%s", nacionalidade);
19 }
20
21 void setCarro(Carro* carro, const char modelo[], float motor, Marca marca) {
22     snprintf(carro->modelo, sizeof(carro->modelo), "%s", modelo);
23     carro->motor = motor;
24     carro->marca = marca; //structs podem ser copiadas por atribuicao
25 }
26
27 void printCarro(Carro carro) {
28     printf("%s\n", carro.modelo);
29     printf("%.2f\n", carro.motor);
30     printf("%s\n", carro.marca.nacionalidade);
31     printf("%s\n", carro.marca.nome);
32 }
33
34 int main(void) {
35     Marca ford;
36     Marca vw;
37     setMarca(&ford, "Ford", "EUA");
38     setMarca(&vw, "VW", "Alemanha");
39     Carro gol;
40     setCarro(&gol, "Gol", 1.0f, vw);
41     printCarro(gol);
42     Carro ka;
43     setCarro(&ka, "Ka", 1.5f, ford);
44     printCarro(ka);
45     return 0;
46 }

```

4.2 Listas Encadeadas

Uma lista encadeada é uma estrutura de dados que permite um crescimento sob demanda, limitado à capacidade de memória disponível no computador. Ela é composta de “nós” que têm suas posições de memória ligadas através de ponteiros. Cada nó da lista, além de guardar dados em si, aponta o próximo nó. Os dados em um nó também podem ser heterogêneos. A Figura 4.1 ilustra essa ideia. Apesar de a figura dar a impressão de que os nós estão distribuídos contiguamente na memória, eles podem estar espalhados em quaisquer endereços, conforme a memória é alocada. Por isso o encadeamento funciona com ponteiros que guardam o endereço de memória do próximo nó da lista. O último nó aponta para NULL, indicando o fim da lista.

Cada nó é criado com a função `malloc`, que reserva memória durante a execução do programa. Dois cuidados são essenciais: reservar o tamanho da *struct* (`sizeof(No)`), e não o



Fig. 4.1: Representação de uma Lista Encadeada.

tamanho do ponteiro, e liberar com `free` cada nó que não for mais usado. Esquecer o `free` provoca vazamento de memória (*memory leak*). A Tabela 4.1 compara o custo das operações em um vetor e em uma lista encadeada com n elementos.

Tabela 4.1: Custo das operações em um vetor e em uma lista simplesmente encadeada com n elementos (CORMEN et al., 2009).

Operação	Vetor	Lista encadeada
Acessar o i -ésimo elemento	$O(1)$	$O(n)$
Buscar um valor	$O(n)$	$O(n)$
Inserir no início	$O(n)$	$O(1)$
Remover um nó já localizado	$O(n)$	$O(1)^*$

*Desde que se conheça o nó anterior.

O Programa 4.6 apresenta uma implementação em C de uma lista encadeada, com funções para inserir no início, buscar, remover e liberar a lista.

Programa 4.6: Implementação em C de lista encadeada.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Define o tipo No contendo um dado e o link para o proximo No
5  typedef struct No {
6      int dado;
7      struct No *link;
8  } No;
9
10 void printFormat01(No* no);
11 void printFormat02(No* no);
12 //Define o primeiro No (cabeca) da lista
13 No* cabeca = NULL;
14
15 //Funcao que adiciona dados no inicio da lista
16 void inserir(int dado) {
17     No* no = malloc(sizeof(No)); //aloca o tamanho da struct, e nao do ponteiro
18     if (no == NULL) {
19         printf("Memoria insuficiente.\n");
20         exit(EXIT_FAILURE);
21     }
22     no->dado = dado;
23     no->link = cabeca; //o novo No aponta para a antiga cabeca (ou NULL)
24     cabeca = no;
25 }
26
27 //Funcao que imprime a lista
28 void imprimirLista(void) {
29     No* no;
30     if (cabeca == NULL) {
31         printf("Lista vazia.\n");
32         return;
33     }
34     no = cabeca;
35     while (no != NULL) {
36         if (no->link != NULL) {
37             printFormat01(no);
38         } else {
39             printFormat02(no);
40         }
41         no = no->link;
42     }
43 }
44
45 void printFormat01(No* no) {
46     printf("[%d(%p) | %p]\n", no->dado, (void *) no, (void *) no->link);
47     printf("          | \n");
48     printf("          V \n");
49     printf("          ----- \n");
50     printf("          | \n");
51     printf("          V \n");
52 }
53
54 void printFormat02(No* no) {
55     printf("[%d(%p) | %p]\n", no->dado, (void *) no, (void *) no->link);
56     printf("          | \n");
57     printf("          V \n");
58     printf("          NULL \n");
59 }
60
61 void buscarDado(int dado) {
62     No* no;
63     if (cabeca == NULL) {
64         printf("Lista vazia.\n");
65         return;
66     }
67     no = cabeca;
68     while (no != NULL) {
69         if (no->dado == dado)

```

```
70         printf("[%d(%p)]\n", no->dado, (void *) no);
71         no = no->link;
72     }
73 }
74
75 void removerDado(int dado) {
76     No *no = cabeca, *anterior = NULL;
77     while (no != NULL) { //lista vazia: o laço não executa
78         if (no->dado == dado) {
79             if (anterior == NULL) { // removendo o primeiro
80                 cabeca = no->link;
81             } else { // removendo do meio ou do fim
82                 anterior->link = no->link; //refaz links
83             }
84             free(no); // libera memória
85             return;
86         }
87         anterior = no; // continua procurando na lista
88         no = no->link;
89     }
90 }
91
92 //Função que libera todos os Nos da lista
93 void liberarLista(void) {
94     while (cabeca != NULL) {
95         No *proximo = cabeca->link;
96         free(cabeca);
97         cabeca = proximo;
98     }
99 }
100
101 int main(void) {
102     // Insere na lista os números de 1 a 4
103     for (int i = 1; i <= 4; i++)
104         inserir(i);
105     imprimirLista();
106     removerDado(2);
107     printf("-----\n");
108     imprimirLista();
109     buscarDado(3);
110     liberarLista();
111     return 0;
112 }
```

As listas encadeadas podem conter ponteiros para outros nós além do próximo. É possível criar uma lista com *links* simultâneos para o próximo nó e para o nó anterior. Essas listas são conhecidas como listas duplamente encadeadas. O Programa 4.7 apresenta uma implementação em C de uma lista duplamente encadeada.

Programa 4.7: Implementação em C de lista duplamente encadeada.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Define o tipo No com links para o proximo e para o anterior
5  typedef struct No {
6      struct No *proximo;
7      struct No *anterior;
8      int dado;
9  } No;
10
11 //Define o primeiro No (cabeca) da lista
12 No* cabeca = NULL;
13
14 //Funcao que adiciona dados no inicio da lista
15 void inserir(int dado) {
16     No* no = malloc(sizeof(No)); //aloca o tamanho da struct, e nao do ponteiro
17     if (no == NULL) {
18         printf("Memoria insuficiente.\n");
19         exit(EXIT_FAILURE);
20     }
21     no->dado = dado;
22     no->anterior = NULL; //o novo No sera o primeiro
23     no->proximo = cabeca;
24     if (cabeca != NULL) { //lista nao vazia: a antiga cabeca aponta de volta para o
25         novo No
26         cabeca->anterior = no;
27     }
28     cabeca = no;
29 }
30 //Funcao que imprime a lista
31 void imprimirLista(void) {
32     if (cabeca == NULL) {
33         printf("Lista vazia.\n");
34         return;
35     }
36     No* no = cabeca;
37     while (no != NULL) {
38         printf("|%p|%d(%p)|%p|\n", (void *) no->anterior, no->dado, (void *) no, (void
39             *) no->proximo);
40         no = no->proximo;
41     }
42     printf("\n");
43 }
44 //Funcao que libera todos os Nos da lista
45 void liberarLista(void) {
46     while (cabeca != NULL) {
47         No *proximo = cabeca->proximo;
48         free(cabeca);
49         cabeca = proximo;
50     }
51 }
52
53 int main(void) {
54     // Insere na lista os numeros de 1 a 5
55     for (int i = 1; i <= 5; i++)
56         inserir(i);
57     imprimirLista();
58     liberarLista();
59     return 0;
60 }

```

4.3 Pilhas

4.4 Filas

4.5 Algoritmos de Ordenação

Bubble Sort

O algoritmo da bolha (*Bubble Sort*) é um algoritmo de ordenação em que cada elemento de uma posição i é comparado com o elemento de posição $i + 1$, os quais trocam de posição, se for o caso, para atender à ordenação procurada (crescente ou decrescente). A cada passagem pelo vetor, o maior elemento ainda desordenado “sobe” até a sua posição final, como uma bolha. Se uma passagem inteira terminar sem trocas, o vetor já está ordenado e o algoritmo pode parar. O Programa 4.8 apresenta uma implementação em C do algoritmo *Bubble Sort*, enquanto a Figura 4.2 ilustra seu funcionamento para um vetor de tamanho 4.

Programa 4.8: Implementação em C do algoritmo de ordenação BubbleSort.

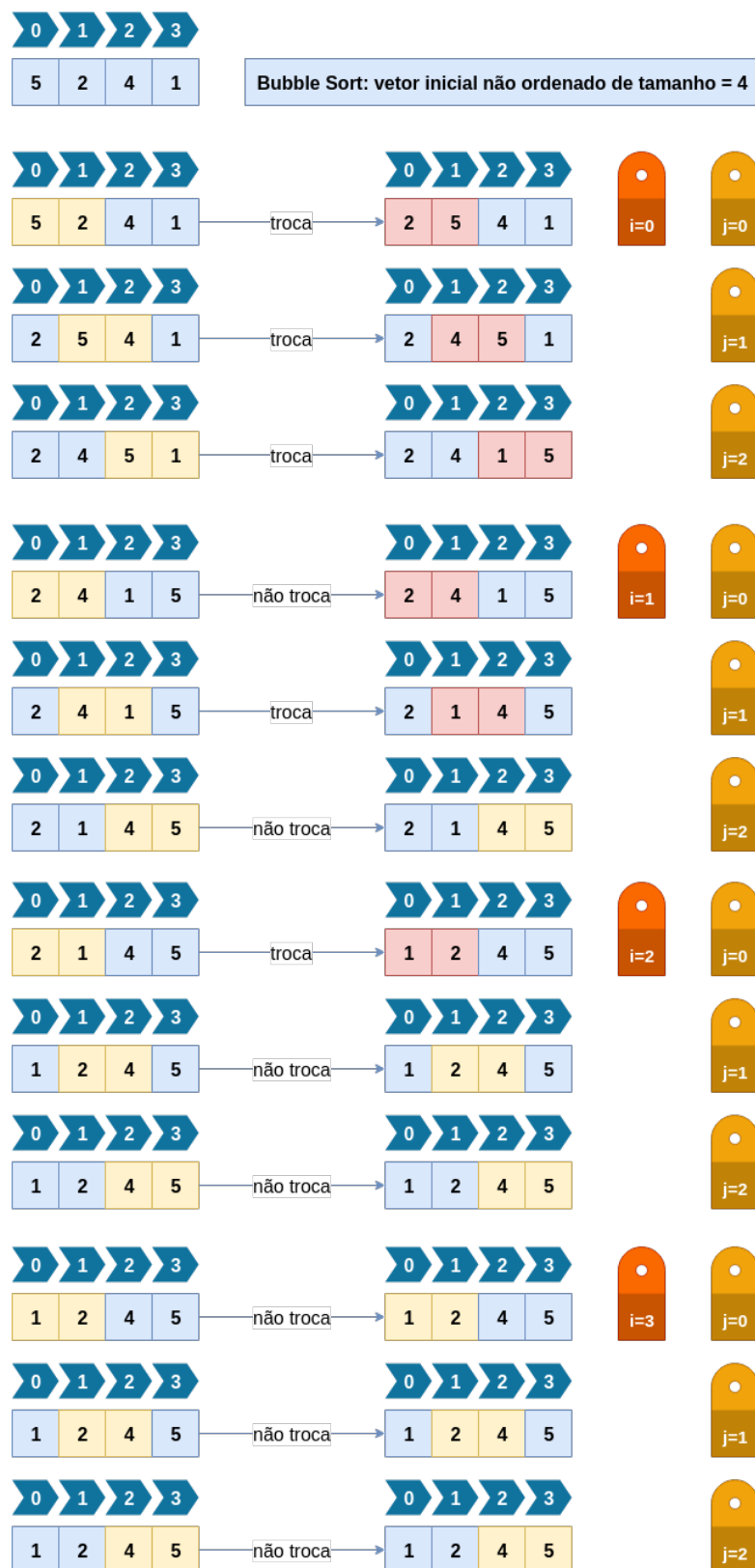
```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define TAMANHO 4
5
6  void bubbleSort(int vetor[], int tamanho);
7  void imprimirVetor(const int vetor[], int tamanho);
8
9  int main(void) {
10     int vetor[TAMANHO]; //vetor com tamanho definido
11     clock_t tempoInicial, tempoFinal; //variaveis para guardar o tempo de execucao
12     srand(42); //semente fixa: mesma sequencia a cada execucao (use srand(time(NULL))
        para variar)
13     for (int i = 0; i < TAMANHO; i++) {
14         vetor[i] = rand() % 10; //atribui um inteiro aleatorio entre 0 e 9
15     }
16     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor nao ordenado
17     tempoInicial = clock(); //inicia contagem do tempo (somente da ordenacao)
18     bubbleSort(vetor, TAMANHO); //ordena pelo metodo da bolha
19     tempoFinal = clock(); //finaliza contagem do tempo
20     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor ordenado
21     //calcula e mostra o tempo total de execucao da ordenacao
22     printf("Tempo: %f s\n", (double) (tempoFinal - tempoInicial) / CLOCKS_PER_SEC);
23     return 0;
24 }
25
26 void imprimirVetor(const int vetor[], int tamanho) {
27     for (int i = 0; i < tamanho; i++) {
28         printf("%d\t", vetor[i]);
29     }
30     printf("\n");
31 }
32
33 void bubbleSort(int vetor[], int tamanho) {
34     int aux; //variavel usada na troca
35     int houveTroca = 1; //indica se alguma troca ocorreu na passagem
36     //a cada passagem, o maior elemento ainda desordenado "sobe" para o fim do vetor
37     for (int i = 0; i < tamanho - 1 && houveTroca; i++) {
38         houveTroca = 0;
39         //os ultimos i elementos ja estao em sua posicao final
40         for (int j = 0; j < tamanho - 1 - i; j++) {
41             if (vetor[j] > vetor[j + 1]) {
42                 aux = vetor[j];
43                 vetor[j] = vetor[j + 1];
44                 vetor[j + 1] = aux;
45                 houveTroca = 1;
46             }
47         }
48     } //se uma passagem inteira nao fez trocas, o vetor ja esta ordenado
49 }

```

Exercícios

1. Atualizar a implementação do *Bubble Sort* para executar as seguintes tarefas:
 - executar o algoritmo com três vetores de entrada:
 - (a) um vetor ordenado crescente de tamanho 100000 (cem mil)
 - (b) um vetor com números aleatórios de tamanho 100000 (cem mil)
 - (c) um vetor ordenado decrescente de tamanho 100000 (cem mil)
 - criar variáveis para contar a quantidade de comparações e quantidade de trocas;
2. Criar gráficos para as três execuções comparando quantidade de comparações e quantidade de trocas e o tempo de execução;

Fig. 4.2: Algoritmo *Bubble Sort* para um vetor de tamanho 4.

Selection Sort

No algoritmo de ordenação por seleção (*Selection Sort*) cada número do vetor, da primeira à penúltima posição, é eleito e comparado com o menor número¹ entre aqueles que estão à direita do eleito. Se o menor for menor que o eleito, os dois trocam de posição. O Programa 4.9 apresenta uma implementação em C do algoritmo *Selection Sort* para um vetor de tamanho 4.

Programa 4.9: Implementação em C do algoritmo de ordenação Selection.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define TAMANHO 4
5
6  void selectionSort(int vetor[], int tamanho);
7  void imprimirVetor(const int vetor[], int tamanho);
8
9  int main(void) {
10     int vetor[TAMANHO]; //vetor com tamanho definido
11     clock_t tempoInicial, tempoFinal; //variaveis para guardar o tempo de execucao
12     srand(42); //semente fixa: mesma sequencia a cada execucao (use srand(time(NULL))
        para variar)
13     for (int i = 0; i < TAMANHO; i++) {
14         vetor[i] = rand() % 10; //atribui um inteiro aleatorio entre 0 e 9
15     }
16     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor nao ordenado
17     tempoInicial = clock(); //inicia contagem do tempo (somente da ordenacao)
18     selectionSort(vetor, TAMANHO); //ordena pelo metodo da selecao
19     tempoFinal = clock(); //finaliza contagem do tempo
20     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor ordenado
21     //calcula e mostra o tempo total de execucao da ordenacao
22     printf("Tempo: %f s\n", (double) (tempoFinal - tempoInicial) / CLOCKS_PER_SEC);
23     return 0;
24 }
25
26 void imprimirVetor(const int vetor[], int tamanho) {
27     for (int i = 0; i < tamanho; i++) {
28         printf("%d\t", vetor[i]);
29     }
30     printf("\n");
31 }
32
33 void selectionSort(int vetor[], int tamanho) {
34     int eleito, posicaoDoMenor;
35     for (int i = 0; i < tamanho - 1; i++) { //da posicao 0 ate a penultima
36         eleito = vetor[i];
37         posicaoDoMenor = i;
38         //procura o menor valor a direita da posicao i
39         for (int j = i + 1; j < tamanho; j++) {
40             if (vetor[j] < vetor[posicaoDoMenor]) {
41                 posicaoDoMenor = j;
42             }
43         }
44         if (posicaoDoMenor != i) { //troca o eleito com o menor encontrado
45             vetor[i] = vetor[posicaoDoMenor];
46             vetor[posicaoDoMenor] = eleito;
47         }
48     }
49 }

```

Exercícios

1. Atualizar a implementação do *Selection Sort* para executar as seguintes tarefas:
 - executar o algoritmo com três vetores de entrada:
 - (a) um vetor ordenado crescente de tamanho 100000 (cem mil)

¹Ou maior, dependendo da ordenação desejada.

-
- (b) um vetor com números aleatórios de tamanho 100000 (cem mil)
 - (c) um vetor ordenado decrescente de tamanho 100000 (cem mil)
 - criar variáveis para contar a quantidade de comparações e quantidade de trocas;
2. Criar gráficos para as três execuções comparando quantidade de comparações e quantidade de trocas e o tempo de execução;

Insertion Sort

O algoritmo de ordenação por inserção (*Insertion Sort*) funciona como a organização de cartas de baralho na mão, em que cada nova carta é inserida na posição correta entre as que já estão ordenadas. No *Insertion Sort*, é eleito o segundo elemento do vetor para iniciar a comparação. Os elementos à esquerda do eleito que forem maiores que ele são deslocados uma posição para a direita, e o eleito é inserido no espaço aberto. O processo se repete para cada elemento seguinte, de forma que a parte à esquerda do eleito esteja sempre ordenada. O Programa 4.10 apresenta uma implementação em C do algoritmo *Insertion Sort* para um vetor de tamanho 4.

Programa 4.10: Implementação em C do algoritmo de ordenação Insertion Sort.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define TAMANHO 4
5
6  void insertionSort(int vetor[], int tamanho);
7  void imprimirVetor(const int vetor[], int tamanho);
8
9  int main(void) {
10     int vetor[TAMANHO]; //vetor com tamanho definido
11     clock_t tempoInicial, tempoFinal; //variaveis para guardar o tempo de execucao
12     srand(42); //semente fixa: mesma sequencia a cada execucao (use srand(time(NULL))
        para variar)
13     for (int i = 0; i < TAMANHO; i++) {
14         vetor[i] = rand() % 10; //atribui um inteiro aleatorio entre 0 e 9
15     }
16     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor nao ordenado
17     tempoInicial = clock(); //inicia contagem do tempo (somente da ordenacao)
18     insertionSort(vetor, TAMANHO); //ordena pelo metodo da insercao
19     tempoFinal = clock(); //finaliza contagem do tempo
20     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor ordenado
21     //calcula e mostra o tempo total de execucao da ordenacao
22     printf("Tempo: %f s\n", (double) (tempoFinal - tempoInicial) / CLOCKS_PER_SEC);
23     return 0;
24 }
25
26 void imprimirVetor(const int vetor[], int tamanho) {
27     for (int i = 0; i < tamanho; i++) {
28         printf("%d\t", vetor[i]);
29     }
30     printf("\n");
31 }
32
33 void insertionSort(int vetor[], int tamanho) {
34     int eleito, j;
35     for (int i = 1; i < tamanho; i++) { //o eleito comeca pelo segundo elemento
36         eleito = vetor[i];
37         j = i - 1;
38         //desloca para a direita os elementos maiores que o eleito
39         while (j >= 0 && vetor[j] > eleito) {
40             vetor[j + 1] = vetor[j];
41             j--;
42         }
43         vetor[j + 1] = eleito; //insere o eleito na posicao correta
44     }
45 }

```

Exercícios

- Atualizar a implementação do *Insertion Sort* para executar as seguintes tarefas:
 - executar o algoritmo com três vetores de entrada:
 - um vetor ordenado crescente de tamanho 100000 (cem mil)
 - um vetor com números aleatórios de tamanho 100000 (cem mil)

-
- (c) um vetor ordenado decrescente de tamanho 100000 (cem mil)
 - criar variáveis para contar a quantidade de comparações e quantidade de trocas;
 - 2. Criar gráficos para as três execuções comparando quantidade de comparações e quantidade de trocas e o tempo de execução;

Merge Sort

O algoritmo de ordenação *Merge Sort* (ordenação por intercalação) usa a técnica de divisão e conquista, ou seja, o problema é dividido em subproblemas menores até que a solução para o subproblema seja simples. As soluções dos subproblemas são combinadas para prover a solução do problema original. No *Merge Sort*, o vetor é dividido ao meio recursivamente até que cada parte tenha apenas um elemento, que já está ordenado por definição. Por exemplo, se um vetor tem n elementos, ele é dividido em duas partes de $n/2$ elementos, que são novamente divididas ao meio, e assim por diante. Na volta da recursão, cada par de partes ordenadas é intercalado (função *merge*): os menores elementos das duas partes são comparados e copiados em ordem, formando uma parte maior também ordenada, até que o vetor original esteja todo ordenado. O Programa 4.11 apresenta uma implementação em C do algoritmo *Merge Sort* para um vetor de tamanho 4.

Programa 4.11: Implementação em C do algoritmo de ordenação Merge Sort.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define TAMANHO 4
5
6  void merge(int vetor[], int inicio, int meio, int fim);
7  void mergeSort(int vetor[], int inicio, int fim);
8  void imprimirVetor(const int vetor[], int tamanho);
9
10 int main(void) {
11     int vetor[TAMANHO]; //vetor com tamanho definido
12     clock_t tempoInicial, tempoFinal; //variaveis para guardar o tempo de execucao
13     srand(42); //semente fixa: mesma sequencia a cada execucao (use srand(time(NULL))
        para variar)
14     for (int i = 0; i < TAMANHO; i++) {
15         vetor[i] = rand() % 10; //atribui um inteiro aleatorio entre 0 e 9
16     }
17     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor nao ordenado
18     tempoInicial = clock(); //inicia contagem do tempo (somente da ordenacao)
19     mergeSort(vetor, 0, TAMANHO - 1); //ordena pelo metodo da intercalacao
20     tempoFinal = clock(); //finaliza contagem do tempo
21     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor ordenado
22     //calcula e mostra o tempo total de execucao da ordenacao
23     printf("Tempo: %f s\n", (double) (tempoFinal - tempoInicial) / CLOCKS_PER_SEC);
24     return 0;
25 }
26
27 void imprimirVetor(const int vetor[], int tamanho) {
28     for (int i = 0; i < tamanho; i++) {
29         printf("%d\t", vetor[i]);
30     }
31     printf("\n");
32 }
33
34 //intercala as metades ordenadas vetor[inicio..meio] e vetor[meio+1..fim]
35 void merge(int vetor[], int inicio, int meio, int fim) {
36     int i, j, k;
37     int n1 = meio - inicio + 1;
38     int n2 = fim - meio;
39     int vetorEsquerda[n1], vetorDireita[n2]; //copias temporarias das duas metades
40     for (i = 0; i < n1; i++)
41         vetorEsquerda[i] = vetor[inicio + i];
42     for (j = 0; j < n2; j++)
43         vetorDireita[j] = vetor[meio + 1 + j];
44     i = 0;
45     j = 0;
46     k = inicio;
47     while (i < n1 && j < n2) { //copia sempre o menor dos dois primeiros
48         if (vetorEsquerda[i] <= vetorDireita[j]) { //"<=" mantem a ordenacao estavel
49             vetor[k] = vetorEsquerda[i];
50             i++;

```

```
51     } else {
52         vetor[k] = vetorDireita[j];
53         j++;
54     }
55     k++;
56 }
57 while (i < n1) { //copia o que restou da metade esquerda
58     vetor[k] = vetorEsquerda[i];
59     i++;
60     k++;
61 }
62 while (j < n2) { //copia o que restou da metade direita
63     vetor[k] = vetorDireita[j];
64     j++;
65     k++;
66 }
67 }
68
69 void mergeSort(int vetor[], int inicio, int fim) {
70     if (inicio < fim) { //condicao de parada: subvetor com um unico elemento
71         int meio = inicio + (fim - inicio) / 2; //posicao para dividir o vetor
72         mergeSort(vetor, inicio, meio); //chamada recursiva para a metade esquerda
73         mergeSort(vetor, meio + 1, fim); //chamada recursiva para a metade direita
74         merge(vetor, inicio, meio, fim); //intercala as duas metades ordenadas
75     }
76 }
```

Exercícios

1. Atualizar a implementação do *Merge Sort* para executar as seguintes tarefas:
 - executar o algoritmo com três vetores de entrada:
 - (a) um vetor ordenado crescente de tamanho 100000 (cem mil)
 - (b) um vetor com números aleatórios de tamanho 100000 (cem mil)
 - (c) um vetor ordenado decrescente de tamanho 100000 (cem mil)
 - criar variáveis para contar a quantidade de comparações e quantidade de trocas;
2. Criar gráficos para as três execuções comparando quantidade de comparações e quantidade de trocas e o tempo de execução;

Quick Sort

O algoritmo de ordenação *Quick Sort* também utiliza a técnica de divisão e conquista. No *Quick Sort*, um elemento do vetor é escolhido como pivô (no Programa 4.12, o primeiro elemento). Em seguida, o vetor é particionado: os valores menores ou iguais ao pivô são levados para a sua esquerda e os maiores para a sua direita. Após esse procedimento, o pivô estará em sua posição definitiva no vetor ordenado. O mesmo processo é aplicado recursivamente às partes à esquerda e à direita do pivô, até que cada parte tenha no máximo um elemento. Diferentemente do *Merge Sort*, todo o trabalho é feito antes das chamadas recursivas, e não há etapa de combinação (HOARE, 1962). O Programa 4.12 apresenta uma implementação em C do algoritmo *Quick Sort* para um vetor de tamanho 4.

Programa 4.12: Implementação em C do algoritmo de ordenação Quick Sort.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4  #define TAMANHO 4
5
6  void trocar(int *x, int *y);
7  void quickSort(int vetor[], int inicio, int fim);
8  void imprimirVetor(const int vetor[], int tamanho);
9
10 int main(void) {
11     int vetor[TAMANHO]; //vetor com tamanho definido
12     clock_t tempoInicial, tempoFinal; //variaveis para guardar o tempo de execucao
13     srand(42); //semente fixa: mesma sequencia a cada execucao (use srand(time(NULL))
        para variar)
14     for (int i = 0; i < TAMANHO; i++) {
15         vetor[i] = rand() % 10; //atribui um inteiro aleatorio entre 0 e 9
16     }
17     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor nao ordenado
18     tempoInicial = clock(); //inicia contagem do tempo (somente da ordenacao)
19     quickSort(vetor, 0, TAMANHO - 1); //ordena pelo metodo rapido
20     tempoFinal = clock(); //finaliza contagem do tempo
21     imprimirVetor(vetor, TAMANHO); //mostra valores do vetor ordenado
22     //calcula e mostra o tempo total de execucao da ordenacao
23     printf("Tempo: %f s\n", (double) (tempoFinal - tempoInicial) / CLOCKS_PER_SEC);
24     return 0;
25 }
26
27 void imprimirVetor(const int vetor[], int tamanho) {
28     for (int i = 0; i < tamanho; i++) {
29         printf("%d\t", vetor[i]);
30     }
31     printf("\n");
32 }
33
34 void trocar(int *x, int *y) { //troca os valores usando ponteiros
35     int aux = *x;
36     *x = *y;
37     *y = aux;
38 }
39
40 void quickSort(int vetor[], int inicio, int fim) {
41     int pivo, i, j;
42     if (inicio < fim) { //condicao de parada
43         pivo = inicio; //o primeiro elemento e escolhido como pivo
44         i = inicio;
45         j = fim;
46         while (i < j) {
47             while (vetor[i] <= vetor[pivo] && i < fim) //avanca enquanto menor ou
                igual ao pivo
48                 i++;
49             while (vetor[j] > vetor[pivo]) //recua enquanto maior que o pivo
50                 j--;
51             if (i < j) {
52                 trocar(&vetor[i], &vetor[j]);

```

```
53     }  
54 }  
55 trocar(&vetor[pivo], &vetor[j]); //coloca o pivo em sua posicao final  
56 quickSort(vetor, inicio, j - 1); //ordena os menores que o pivo  
57 quickSort(vetor, j + 1, fim); //ordena os maiores que o pivo  
58 }  
59 }
```

Exercícios

1. Atualizar a implementação do *Quick Sort* para executar as seguintes tarefas:
 - executar o algoritmo com três vetores de entrada:
 - (a) um vetor ordenado crescente de tamanho 100000 (cem mil)
 - (b) um vetor com números aleatórios de tamanho 100000 (cem mil)
 - (c) um vetor ordenado decrescente de tamanho 100000 (cem mil)
 - criar variáveis para contar a quantidade de comparações e quantidade de trocas;
2. Criar gráficos para as três execuções comparando quantidade de comparações e quantidade de trocas e o tempo de execução;

4.6 Árvores

Uma árvore binária é uma estrutura em que cada nó tem no máximo dois filhos, chamados de filho esquerdo e filho direito. O nó sem pai é a raiz, e os nós sem filhos são as folhas. Em uma árvore binária de busca, para todo nó, os valores da subárvore esquerda são menores que o valor do nó, e os da subárvore direita são maiores (CORMEN et al., 2009). Essa propriedade permite buscar um valor descendo pela árvore a partir da raiz, em tempo proporcional à altura da árvore. O Programa 4.13 apresenta uma implementação em C de uma árvore binária de busca, com inserção, busca, cálculo da altura e liberação da memória. O percurso em ordem simétrica (esquerda, raiz, direita) mostra os valores em ordem crescente. O arquivo `ArvoreBinaria_com_impressao.c`, no mesmo diretório do repositório, gera também uma representação gráfica da árvore no formato da ferramenta Graphviz (ELLSON et al., 2004).

Programa 4.13: Implementação em C de árvore binária de busca.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef struct No {
5      int dado;
6      struct No* direita;
7      struct No* esquerda;
8  } No;
9
10 No* criarArvore(void) { return NULL; }
11
12 int arvoreVazia(No* raiz) { // 1 se a arvore esta vazia, 0 caso contrario
13     return raiz == NULL;
14 }
15
16 // Percurso em ordem simetrica (esquerda, raiz, direita): mostra os dados em ordem
17 // crescente
18 void mostrarArvore(No* raiz) {
19     if (!arvoreVazia(raiz)) {
20         mostrarArvore(raiz->esquerda);
21         printf("%d ", raiz->dado);
22         mostrarArvore(raiz->direita);
23     }
24 }
25
26 // Devolve 1 se o dado estiver na arvore e 0 caso contrario
27 int buscarDado(No* raiz, int dado) {
28     if (arvoreVazia(raiz))
29         return 0; // chegou a uma folha sem encontrar
30     if (dado == raiz->dado)
31         return 1;
32     if (dado < raiz->dado) // dado menor? vai pra esquerda
33         return buscarDado(raiz->esquerda, dado);
34     if (dado > raiz->dado) // dado maior? vai pra direita
35         return buscarDado(raiz->direita, dado);
36 }
37
38 // Recebe o endereco do ponteiro para poder alterar a arvore do chamador
39 void inserirDado(No** raiz, int dado) {
40     if (*raiz == NULL) {
41         *raiz = malloc(sizeof(No));
42         if (*raiz == NULL) {
43             printf("Memoria insuficiente.\n");
44             exit(EXIT_FAILURE);
45         }
46         (*raiz)->esquerda = NULL;
47         (*raiz)->direita = NULL;
48         (*raiz)->dado = dado;
49     } else if (dado < (*raiz)->dado) { // dado menor? vai pra esquerda
50         inserirDado(&(*raiz)->esquerda, dado);
51     } else if (dado > (*raiz)->dado) { // dado maior? vai pra direita
52         inserirDado(&(*raiz)->direita, dado);
53     } // dado igual: nao insere repetidos

```

```

52 }
53
54 int getAltura(No* raiz) {
55     if (raiz == NULL) return -1; // altura da arvore vazia
56     int hEsquerda = getAltura(raiz->esquerda);
57     int hDireita = getAltura(raiz->direita);
58     return (hEsquerda < hDireita) ? hDireita + 1 : hEsquerda + 1;
59 }
60
61 // Libera os nos em pos-ordem: primeiro os filhos, depois o pai
62 void liberarArvore(No* raiz) {
63     if (raiz != NULL) {
64         liberarArvore(raiz->esquerda);
65         liberarArvore(raiz->direita);
66         free(raiz);
67     }
68 }
69
70 int main(void) {
71     No* raiz = criarArvore();
72     srand(42); // semente fixa: mesma arvore a cada execucao
73     for (int i = 0; i < 50; i++) {
74         inserirDado(&raiz, rand() % 100);
75     }
76     mostrarArvore(raiz);
77     printf("\n");
78     printf("7 %s\n", buscarDado(raiz, 7) ? "encontrado." : "nao encontrado.");
79     printf("Altura: %d\n", getAltura(raiz));
80     liberarArvore(raiz);
81     return 0;
82 }

```

4.7 Tabelas Hashing

4.8 Grafos

Grafos são estruturas de dados muito poderosas para encontrar soluções para diversos problemas. Exemplos disso são os algoritmos de cálculo de rotas, como os de aplicativos de navegação por GPS, ou ainda a montagem de genomas a partir de fragmentos de DNA sequenciados (PEVZNER; TANG; WATERMAN, 2001). É na verdade muito expressiva a quantidade de problemas do mundo real que podem ser computacionalmente modelados como grafos.

Um grafo $G = (V, E)$ é formado por dois conjuntos V e E , onde:

- V é um conjunto finito e não vazio de vértices de G ;
- E é um conjunto finito de arestas de G .

Uma aresta $e \in E(G)$ é representada por $e = (v_1, v_2)$ ligando os vértices v_1 e v_2 . Vejamos um exemplo. Seja o conjunto $V = \{a, b, c, d, e\}$. Seja o conjunto $E = \{(a, b), (a, c), (b, c), (b, d), (c, d), (c, e), (d, e)\}$. Uma representação gráfica deste grafo pode ser vista na Figura 4.3.

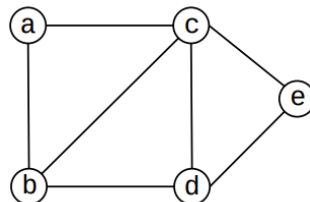


Fig. 4.3: Exemplo de grafo. Fonte: <https://www.ic.unicamp.br/~zanoni/teaching/>

Dada uma aresta $e = (a, b)$, os vértices a e b são os “extremos”² da aresta e . Os vértices a e

² a e b estão nas extremidades de e .

b são também vértices adjacentes. Além disso, a aresta e é incidente aos vértices a e b .

Denota-se por $|V|$ e $|E|$ a cardinalidade dos conjuntos de $V(G)$ e $E(G)$. No exemplo da Figura 4.3, teríamos: $|V| = 5$ e $|E| = 7$. O tamanho do grafo é dado por $|V| + |E|$.

Grafos Simples

Um grafo é dito simples, se não possui laços ou arestas múltiplas. Um laço é uma aresta com extremos idênticos. Arestas múltiplas são duas ou mais arestas com os mesmos extremos. A Figura 4.4 mostra um exemplo de laço e arestas múltiplas.

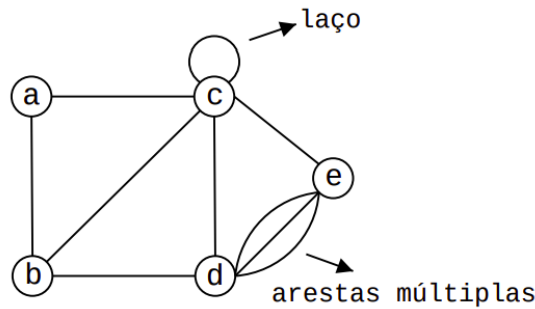


Fig. 4.4: Exemplo de grafo com laços e arestas múltiplas. Fonte: <https://www.ic.unicamp.br/~zanoni/teaching/>

Subgrafo e Subgrafo Gerador

Um subgrafo $G' = (V', E')$ de um grafo $G = (V, E)$ é um grafo tal que $V' \subseteq V, E' \subseteq E$. Diz-se que um subgrafo $G' = (V', E')$ de um grafo $G = (V, E)$ é gerador se $V' = V$. A Figura 4.5 mostra exemplos de subgrafo e subgrafo gerador.

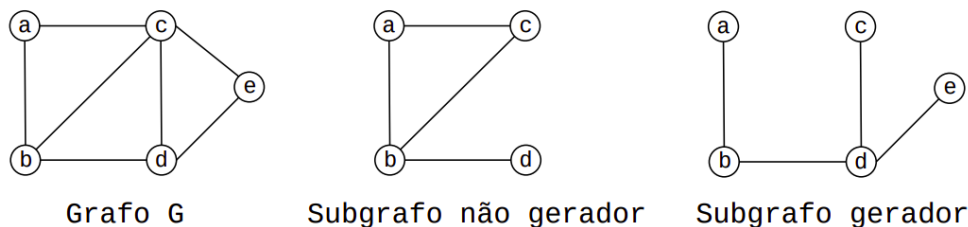


Fig. 4.5: Exemplo de grafo, subgrafo e subgrafo gerador. Fonte: <https://www.ic.unicamp.br/~zanoni/teaching/>

Grau de um Vértice

O grau (*degree*) de um vértice $v \in V(G)$, denotado por $d(v)$, é igual ao número de arestas incidentes a v . Laços são contados em dobro. A Figura 4.6 traz um exemplo para graus de vértices de um grafo. Uma interessante propriedade, conhecida como *Handshaking lemma* (lema do aperto de mãos), depende-se de $d(v)$: como cada aresta tem dois extremos, ela contribui com duas unidades para a soma dos graus. A soma dos graus dos vértices é igual a duas vezes a cardinalidade do conjunto de arestas:

$$\sum_{v \in V} d(v) = 2|E|$$

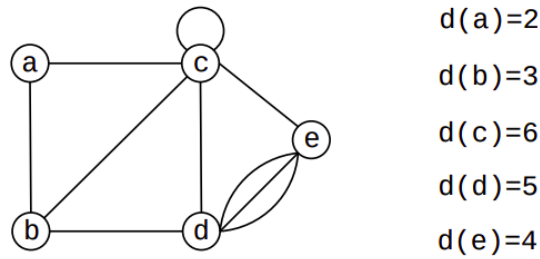


Fig. 4.6: Exemplo de graus de vértices em um grafo. Fonte: <https://www.ic.unicamp.br/~zanoni/teaching/>

Grafo Direcionado ou Orientado

Um grafo $G = (V, E)$ é orientado, ou direcionado (digrafo), quando as arestas do conjunto E estão dispostas em “pares ordenados”, ou seja, existe a conotação de direção em suas extremidades. Em um digrafo, o grau de entrada de um vértice é o número de arestas que chegam a ele, e o grau de saída é o número de arestas que partem dele. Por exemplo, seja a aresta $e_1 = (v_1, v_2)$. Se e_1 sai de v_1 e entra em v_2 , temos uma direção definida de v_1 para v_2 . A Figura 4.7 mostra um exemplo de grafo orientado.

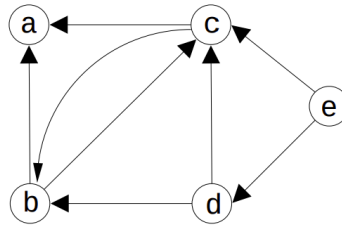


Fig. 4.7: Exemplo de grafo orientado. Fonte: <https://www.ic.unicamp.br/~zanoni/teaching/>

Representação de Grafos

Apesar de a representação visual da informação ser uma das vantagens dos grafos, armazenar computacionalmente os grafos como figuras limita a aplicação dos algoritmos em grafos. Portanto, outras maneiras de abstrair os grafos foram pensadas. As mais largamente utilizadas são as listas e as matrizes de adjacência.

Suponha que um grafo tenha n vértices numerados $v_1, v_2, \dots, v_n$ apenas para fins de identificação (a numeração não implica nenhuma ordem entre os vértices). Uma matriz de adjacência é uma matriz $n \times n$ em que o elemento (i, j) é o número de arestas entre os vértices v_i e v_j . Tomando como exemplo o grafo da Figura 4.3, a matriz de adjacência seria uma matriz 5×5 , já que temos nele o conjunto $V = \{a, b, c, d, e\}$ e o conjunto $E = \{(a, b), (a, c), (b, c), (b, d), (c, d), (c, e), (d, e)\}$.

Para grafos não orientados, a matriz de adjacência é simétrica, de forma que nesses casos apenas é necessário o armazenamento dos elementos da diagonal principal e dos elementos abaixo (ou acima) dela. Já a matriz de um grafo direcionado não será simétrica porque a existência de uma aresta de n_i para n_j não implica na existência de uma aresta de n_j para n_i . Os valores da matriz podem ser os pesos das arestas. A Figura 4.8 mostra a matriz de adjacência do grafo da Figura 4.3 completa e na forma triangular, que aproveita a simetria.

$$\left(\begin{array}{c|ccccc} & a & b & c & d & e \\ \hline a & 0 & 1 & 1 & 0 & 0 \\ b & 1 & 0 & 1 & 1 & 0 \\ c & 1 & 1 & 0 & 1 & 1 \\ d & 0 & 1 & 1 & 0 & 1 \\ e & 0 & 0 & 1 & 1 & 0 \end{array} \right) \quad \left(\begin{array}{c|ccccc} & a & b & c & d & e \\ \hline a & \mathbf{0} & & & & \\ b & \mathbf{1} & \mathbf{0} & & & \\ c & \mathbf{1} & \mathbf{1} & \mathbf{0} & & \\ d & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{0} & \\ e & \mathbf{0} & \mathbf{0} & \mathbf{1} & \mathbf{1} & \mathbf{0} \end{array} \right)$$

Fig. 4.8: Matriz de adjacência do grafo não direcionado da Figura 4.3: completa (esquerda) e apenas com a diagonal principal e os elementos abaixo dela (direita).

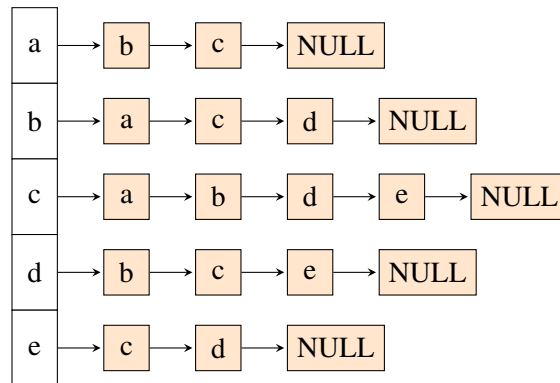


Fig. 4.9: Lista de adjacências do grafo da Figura 4.3.

Quando um grafo tem muitos vértices, mas relativamente poucas arestas, a matriz de adjacências torna-se esparsa. Ainda assim, serão necessários n^2 dados para representar um grafo com n vértices. Consequentemente, o algoritmo para percorrer toda a matriz precisará “olhar” n^2 elementos. Em casos assim, o grafo pode ser melhor representado usando uma lista de adjacências. Em uma lista de adjacências, cada vértice tem uma lista encadeada com os seus vértices adjacentes, e o espaço ocupado é proporcional a $|V| + |E|$ (CORMEN et al., 2009). Tomando o mesmo exemplo do grafo da Figura 4.3, teríamos uma lista de adjacências como mostrada na Figura 4.9.

4.8.1 Grafos como objetos (Programação orientada a objetos)

A programação orientada a objetos permite abstrair, em modelos chamados classes, um novo “tipo” de dados que possa representar uma instância de um problema do “mundo real”. Em se tratando de grafos, os vértices, as arestas e o próprio grafo podem ser objetos das respectivas classes. Uma implementação em Python dessa abordagem, com as classes `Vertice`, `Aresta` e `Grafo` e os algoritmos de busca em profundidade, busca em largura, Prim e Dijkstra, está disponível no diretório [Code/Python](#) do repositório deste material.

4.8.2 Busca em Grafos

A busca em profundidade (*Depth-First Search*, DFS) é um algoritmo para percorrer os vértices de um grafo. Dado um grafo $G = (V, E)$ e um vértice inicial v_1 , a estratégia da busca em profundidade é buscar, sempre que possível, o mais profundo no grafo. Partindo de v_1 , a ideia é seguir por uma aresta até um vértice ainda não visitado e, a partir dele, repetir o processo (ZIVIANI, 2007). Quando todas as arestas do vértice v mais recentemente descoberto tiverem sido

exploradas, a busca anda para trás (*backtrack*) até o vértice a partir do qual v foi descoberto, para explorar as arestas restantes dele (CORMEN et al., 2009). O processo se repete até que todos os vértices alcançáveis a partir de v_1 tenham sido visitados. A busca em profundidade pode ser implementada com recursão ou com uma pilha explícita, que guarda os vértices a visitar. Tipicamente a complexidade de tempo desse algoritmo é de $O(|V| + |E|)$ para percorrer o grafo inteiro representado por listas de adjacências (CORMEN et al., 2009). A Figura 4.10 mostra um exemplo ilustrado do funcionamento do algoritmo de busca em profundidade com uma pilha, para o grafo direcionado com $V = \{A, B, C, D, E\}$ e $E = \{(A, B), (A, C), (A, D), (B, C), (C, E)\}$. Os vértices em verde já foram visitados, os amarelos estão na pilha e os vermelhos ainda não foram descobertos.

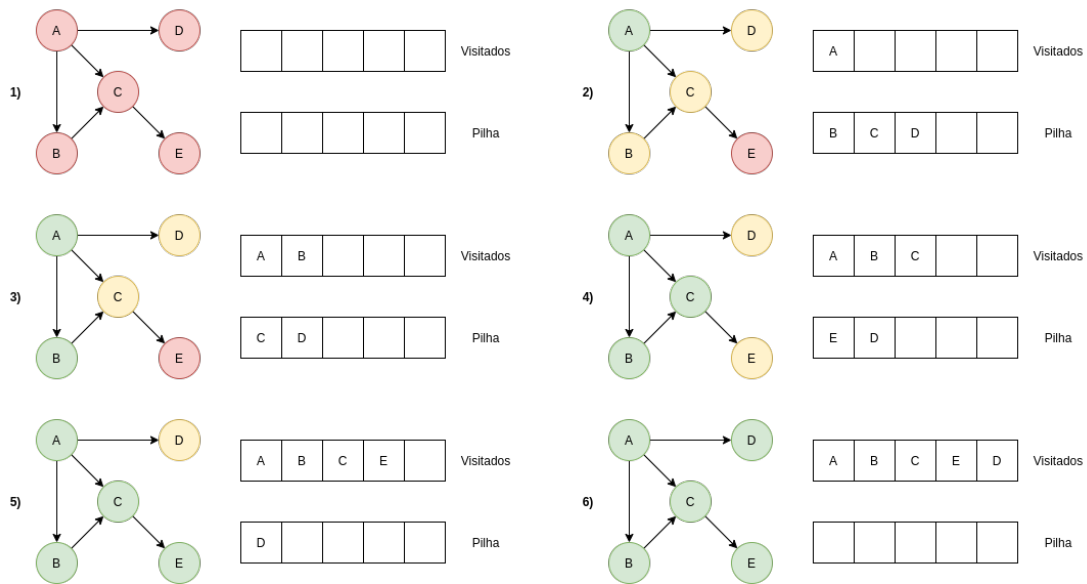


Fig. 4.10: Exemplo de busca em profundidade a partir do vértice A. Ordem de visita: A, B, C, E, D.

A busca em largura (*Breadth-First Search*, BFS) é uma estratégia que expande a fronteira entre vértices descobertos e não descobertos de maneira uniforme: primeiro visita todos os vértices a uma aresta de distância de v_1 , depois os que estão a duas arestas, e assim por diante (CORMEN et al., 2009). Dado um grafo $G = (V, E)$ e um vértice v_1 , inicialmente inclui-se o vértice v_1 em uma lista de vértices visitados. Busca-se então todos os vértices adjacentes a v_1 , os quais são incluídos em uma fila para serem visitados. Os vértices vão sendo incluídos na lista de visitados à medida que vão sendo retirados da fila, e os seus adjacentes ainda não descobertos entram no fim da fila. O mesmo procedimento é executado sucessivamente até que não haja mais nenhum vértice na fila. Em um grafo sem pesos, a busca em largura encontra o caminho com o menor número de arestas entre v_1 e cada vértice alcançável. Tipicamente a complexidade de tempo desse algoritmo também é de $O(|V| + |E|)$ para percorrer o grafo inteiro (CORMEN et al., 2009). A Figura 4.11 mostra um exemplo ilustrado do funcionamento do algoritmo de busca em largura para o mesmo grafo da Figura 4.10.

4.8.3 Caminho de Euler

Um caminho euleriano, ou caminho de Euler, é um caminho em um grafo $G = (V, E)$, direcionado ou não, que percorre **cada aresta exatamente uma vez**, enquanto os vértices podem ser visitados mais de uma vez. Se o caminho de Euler inicia e termina em um mesmo vértice, então esse caminho é chamado de circuito ou ciclo de Euler. Considerando apenas os vértices com pelo

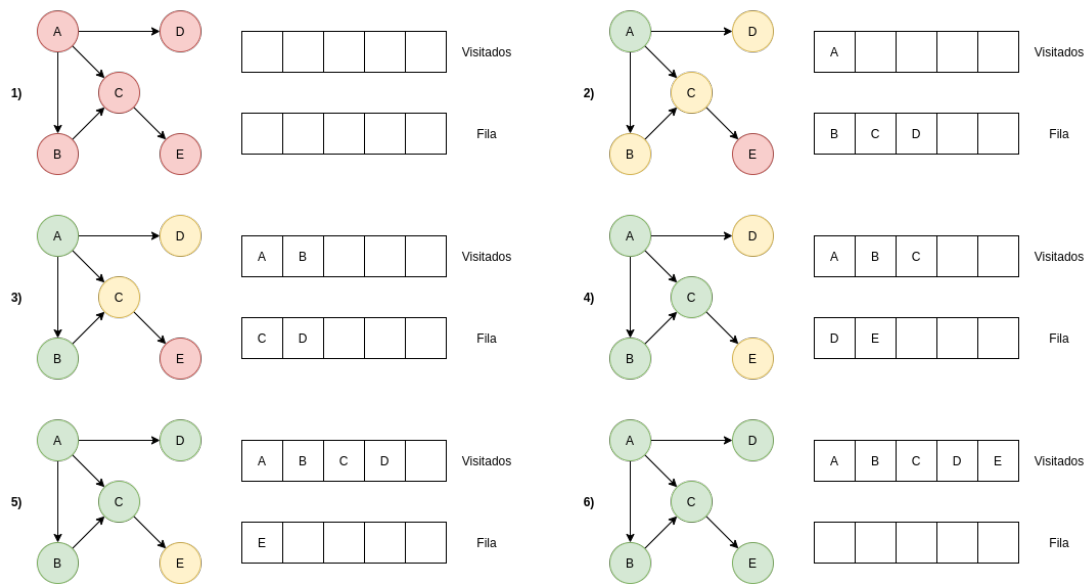


Fig. 4.11: Exemplo de busca em largura a partir do vértice A. Ordem de visita: A, B, C, D, E.

menos uma aresta, e desde que eles formem um grafo conexo, as condições de existência são as seguintes (CORMEN et al., 2009):

- Em um grafo **não direcionado**, existe circuito de Euler se e somente se todos os vértices têm grau par; existe caminho de Euler que não é circuito se e somente se exatamente dois vértices têm grau ímpar, e eles são o início e o fim do caminho.
- Em um grafo **direcionado**, existe circuito de Euler se e somente se, em todo vértice, o grau de entrada é igual ao grau de saída; existe caminho de Euler que não é circuito se e somente se um vértice tem grau de saída igual ao de entrada mais um (o início), um vértice tem grau de entrada igual ao de saída mais um (o fim) e todos os demais têm graus de entrada e de saída iguais.

Diferentemente do caminho hamiltoniano, a existência de um caminho de Euler pode ser verificada, e o caminho construído, em tempo linear no número de arestas. Caminhos de Euler em grafos de De Bruijn são a base de métodos de montagem de genomas (PEVZNER; TANG; WATERMAN, 2001). O diagrama da Figura 4.12 mostra um caminho de Euler para um grafo direcionado. Nele, cada aresta foi visitada uma única vez. O caminho de Euler foi possível porque o vértice A tem grau de saída igual ao de entrada mais um (início do caminho), o vértice C tem grau de entrada igual ao de saída mais um (fim do caminho) e os demais vértices têm graus de entrada e de saída iguais.

4.8.4 Caminho Hamiltoniano

Um caminho hamiltoniano é um caminho em um grafo $G = (V, E)$, direcionado ou não, que visita **cada vértice exatamente uma vez**. O problema de decidir se um grafo contém um caminho hamiltoniano é NP-completo (KARP, 1972; GAREY; JOHNSON, 1979). A classe NP (tempo polinomial não determinístico) reúne os problemas de decisão cujas soluções, uma vez apresentadas, podem ser **verificadas** em tempo polinomial: dada uma sequência de vértices, é fácil conferir se ela é um caminho hamiltoniano. Um problema é NP-completo quando pertence a NP e todo problema de NP pode ser reduzido a ele em tempo polinomial (COOK, 1971). Não se conhece algoritmo de tempo polinomial para nenhum problema NP-completo; encontrar um, ou provar que não existe, é justamente a questão P versus NP citada no Capítulo 2. O diagrama da Figura 4.13 mostra um caminho Hamiltoniano para um grafo não direcionado.

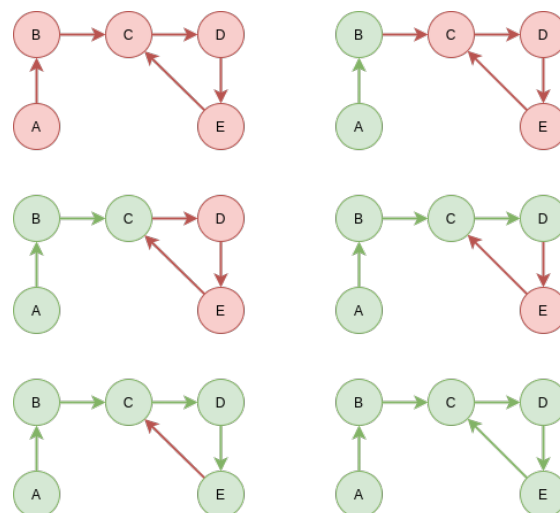


Fig. 4.12: Exemplo de caminho de Euler em um grafo direcionado.

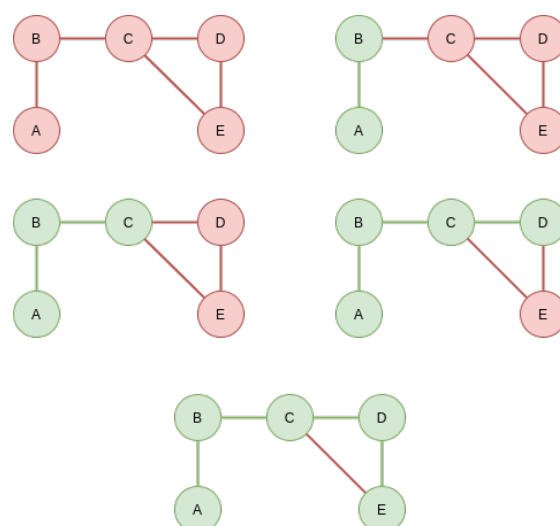
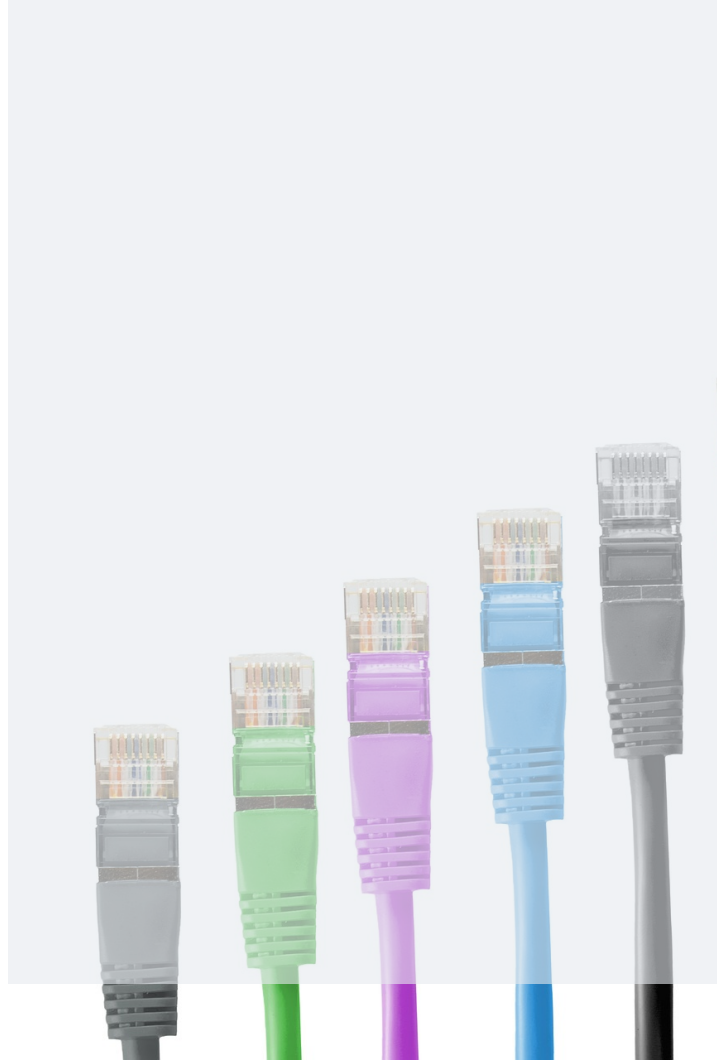
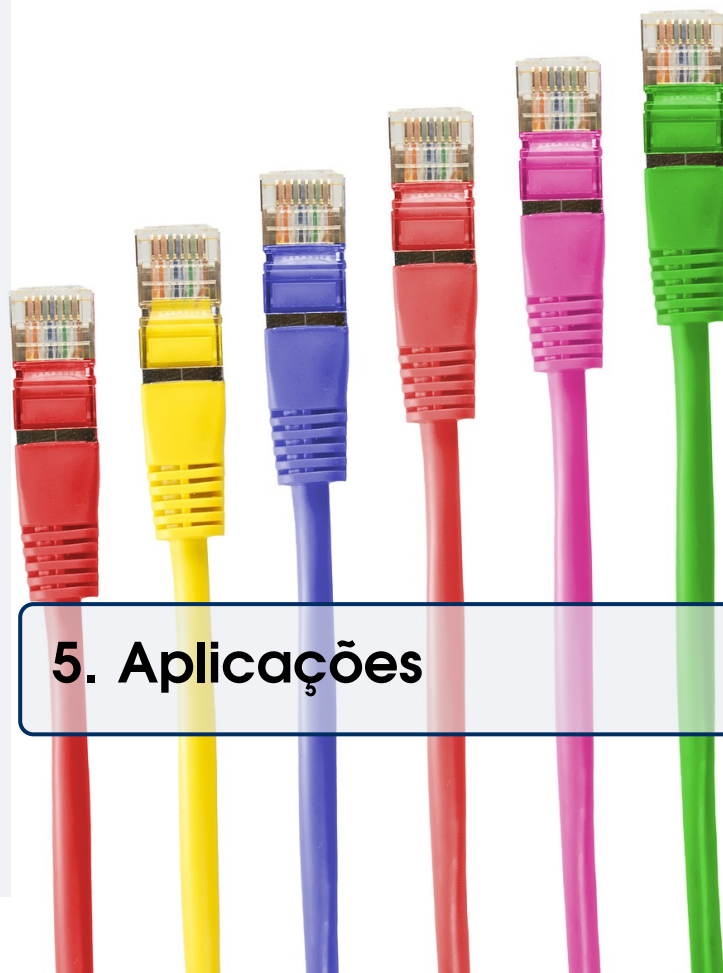


Fig. 4.13: Exemplo de caminho Hamiltoniano em um grafo não direcionado.



5. Aplicações



Referências Bibliográficas

ASCENCIO, Ana Fernanda Gomes; ARAÚJO, Graziela Santos de. **Estruturas de Dados: algoritmos, análise da complexidade e implementações em Java e C/C++**. São Paulo: Pearson Prentice Hall, 2010.

CLAY MATHEMATICS INSTITUTE. **The Millennium Prize Problems**. 2000. Available from: <<https://www.claymath.org/millennium-problems/>>. Visited on: 24 Sept. 2026.

COOK, Stephen A. The Complexity of Theorem-Proving Procedures. In: PROCEEDINGS of the Third Annual ACM Symposium on Theory of Computing. [S.l.: s.n.], 1971. P. 151–158. DOI: [10.1145/800157.805047](https://doi.org/10.1145/800157.805047).

CORMEN, Thomas H. et al. **Introduction to Algorithms**. 3. ed. Cambridge: MIT Press, 2009.

ELLSON, John et al. Graphviz and Dynagraph – Static and Dynamic Graph Drawing Tools. In: JÜNGER, Michael; MUTZEL, Petra (Eds.). **Graph Drawing Software**. Berlin: Springer, 2004. P. 127–148.

GAREY, Michael R.; JOHNSON, David S. **Computers and Intractability: A Guide to the Theory of NP-Completeness**. San Francisco: W. H. Freeman, 1979.

HOARE, C. A. R. Quicksort. **The Computer Journal**, v. 5, n. 1, p. 10–16, 1962. DOI: [10.1093/comjnl/5.1.10](https://doi.org/10.1093/comjnl/5.1.10).

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 9899:2018: Information technology – Programming languages – C**. Genebra, 2018.

KARP, Richard M. Reducibility among Combinatorial Problems. In: MILLER, Raymond E.; THATCHER, James W. (Eds.). **Complexity of Computer Computations**. New York: Plenum Press, 1972. P. 85–103. DOI: [10.1007/978-1-4684-2001-2_9](https://doi.org/10.1007/978-1-4684-2001-2_9).

KNUTH, Donald E. Big Omicron and Big Omega and Big Theta. **ACM SIGACT News**, v. 8, n. 2, p. 18–24, 1976. DOI: [10.1145/1008328.1008329](https://doi.org/10.1145/1008328.1008329).

MANBER, Udi. **Introduction to Algorithms: A Creative Approach**. Reading: Addison-Wesley, 1989.

PEVZNER, Pavel A.; TANG, Haixu; WATERMAN, Michael S. An Eulerian Path Approach to DNA Fragment Assembly. **Proceedings of the National Academy of Sciences**, v. 98, n. 17, p. 9748–9753, 2001. DOI: [10.1073/pnas.171285098](https://doi.org/10.1073/pnas.171285098).

ZIVIANI, Nivio. **Projeto de Algoritmos com Implementações em Java e C++**. São Paulo: Thomson Learning, 2007.

```
1 <!--meta-->
2 <!--meta-->
3 <!--meta-->
4 <!--meta-->
5 <!--meta-->
6 <!--meta-->
7 <!--meta-->
8 <!--meta-->
9 <!--meta-->
10 <!--meta-->
11 <!--meta-->
12 <!--meta-->
13 <!--meta-->
14 <!--meta-->
15 <!--meta-->
16 <!--meta-->
17 <!--meta-->
18 <!--meta-->
19 <!--meta-->
20 <!--meta-->
21 <!--meta-->
22 <!--meta-->
23 <!--meta-->
24 <!--meta-->
25 <!--meta-->
26 <!--meta-->
27 <!--meta-->
28 <!--meta-->
29 <!--meta-->
30 <!--meta-->
31 <!--meta-->
32 <!--meta-->
33 <!--meta-->
34 <!--meta-->
35 <!--meta-->
36 <!--meta-->
37 <!--meta-->
38 <!--meta-->
39 <!--meta-->
40 <!--meta-->
41 <!--meta-->
42 <!--meta-->
43 <!--meta-->
44 <!--meta-->
45 <!--meta-->
46 <!--meta-->
47 <!--meta-->
48 <!--meta-->
49 <!--meta-->
50 <!--meta-->
51 <!--meta-->
52 <!--meta-->
53 <!--meta-->
54 <!--meta-->
55 <!--meta-->
56 <!--meta-->
57 <!--meta-->
58 <!--meta-->
59 <!--meta-->
60 <!--meta-->
61 <!--meta-->
62 <!--meta-->
63 <!--meta-->
64 <!--meta-->
65 <!--meta-->
66 <!--meta-->
67 <!--meta-->
68 <!--meta-->
69 <!--meta-->
70 <!--meta-->
71 <!--meta-->
72 <!--meta-->
73 <!--meta-->
74 <!--meta-->
75 <!--meta-->
76 <!--meta-->
77 <!--meta-->
78 <!--meta-->
79 <!--meta-->
80 <!--meta-->
81 <!--meta-->
82 <!--meta-->
83 <!--meta-->
84 <!--meta-->
85 <!--meta-->
86 <!--meta-->
87 <!--meta-->
88 <!--meta-->
89 <!--meta-->
90 <!--meta-->
91 <!--meta-->
92 <!--meta-->
93 <!--meta-->
94 <!--meta-->
95 <!--meta-->
96 <!--meta-->
97 <!--meta-->
98 <!--meta-->
99 <!--meta-->
100 <!--meta-->
```

6. Apêndice